

FinOps

Documentacao do Banco de Dados

Sistema de Organizacao Financeira Pessoal

Projeto FinOps
Documentacao Tecnica do Banco de Dados

5 de agosto de 2025

Sumário

1	Arquitetura do Banco de Dados	3
1.1	Arquitetura Geral	3
1.2	Distribuicao dos Dados	3
1.2.1	Esquemas Logicos	3
1.3	Relacionamentos Principais	3
1.3.1	Hierarquia de Usuarios	3
1.3.2	Fluxo de Dados Financeiros	4
1.4	Estrategia de Particao	4
1.4.1	Particao por Data	4
1.4.2	Particao por Usuario	4
1.5	Estrategia de Cache	4
1.6	Integracoes Externas	4
2	Modelo Conceitual	5
2.1	Diagrama Entidade-Relacionamento	5
2.1.1	Entidades Principais	5
2.2	Relacionamentos	6
2.2.1	Usuario - Grupo (N:M)	6
2.2.2	Usuario - Conta (1:N)	6
2.2.3	Conta - Transacao (1:N)	6
2.2.4	Categoria - Transacao (1:N)	6
2.2.5	Usuario - Orcamento (1:N)	6
2.3	Cardinalidades e Restricoes	6
2.4	Regras de Negocio	7
2.4.1	Modelos de Organizacao	7
2.4.2	Integridade Referencial	7
3	Modelo Logico	7
3.1	Estrutura das Tabelas	7
3.1.1	Tabela: usuarios	7
3.1.2	Tabela: grupos	8
3.1.3	Tabela: usuario_grupos	8
3.1.4	Tabela: contas	8
3.1.5	Tabela: categorias	9
3.1.6	Tabela: transacoes	10
3.2	Indices Principais	11
3.3	Constraints e Validacoes	11
4	Modelo Fisico	12
4.1	Implementacao Completa	12
4.1.1	Tabelas Complementares	12
4.1.2	Tabelas de Auditoria	14
4.2	Views Materializadas	15
4.3	Funcoes de Sistema	16

5	Scripts de Criacao	17
5.1	Script Principal de Instalacao	17
5.2	Script de Rollback	22
6	Procedures e Funcoes	23
6.1	Funcoes de Negocio	23
6.1.1	Calculo Splitwise	23
6.1.2	Atualizacao de Saldos	24
6.1.3	Processamento de Recorrencias	25
6.2	Funcoes de Relatorio	27
6.2.1	Resumo Financeiro	27
6.3	Funcoes de Validacao	28
6.3.1	Validacao de Limite de Credito	28
7	Triggers	29
7.1	Triggers de Auditoria	29
7.1.1	Trigger de Log Automatico	29
7.1.2	Aplicacao dos Triggers	29
7.2	Triggers de Negocio	30
7.2.1	Atualizacao Automatica de Saldos	30
7.2.2	Validacao de Integridade	31
7.3	Triggers de Performance	31
7.3.1	Atualizacao de Views Materializadas	31
8	Indices e Otimizacao	32
8.1	Estrategia de Indexacao	32
8.1.1	Indices Primarios	32
8.1.2	Indices Condicionais	32
8.2	Monitoramento de Performance	33
8.2.1	Views de Monitoramento	33
8.3	Otimizacoes Avancadas	34
8.3.1	Particao de Tabelas	34
8.3.2	Cache e Buffers	34
8.4	Analise e Manutencao	35
8.4.1	Scripts de Analise	35
8.4.2	Manutencao Automatica	36
9	Seguranca	36
9.1	Controle de Acesso	36
9.1.1	Roles e Permissoes	36
9.1.2	Row Level Security (RLS)	37
9.2	Criptografia	38
9.2.1	Criptografia de Dados Sensiveis	38
9.3	Auditoria e Logging	39
9.3.1	Log de Acessos	39
9.3.2	Deteccao de Anomalias	40
9.4	Configuracoes de Seguranca	41
9.4.1	Parametros do PostgreSQL	41
9.5	Backup Seguro	41

10 Backup e Recovery	42
10.1 Estrategia de Backup	42
10.1.1 Tipos de Backup	42
10.2 Procedures de Recovery	44
10.2.1 Recovery Completo	44
10.2.2 Point-in-Time Recovery	44
10.3 Monitoramento de Backup	45
10.3.1 Verificacao Automatica	45
10.4 Disaster Recovery	46
10.4.1 Plano de Contingencia	46
10.4.2 Replicacao para Alta Disponibilidade	47
11 Performance e Otimizacao	48
11.1 Monitoramento de Performance	48
11.1.1 Metricas Essenciais	48
11.1.2 Dashboard de Monitoramento	49
11.2 Otimizacoes de Query	50
11.2.1 Analise de Queries Lentas	50
11.2.2 Otimizacoes Especificas	51
11.3 Manutencao Preventiva	51
11.3.1 Scripts de Manutencao	51
11.3.2 Monitoramento Automatico	52
11.4 Configuracoes de Performance	53
11.4.1 Parametros Otimizados	53
12 Manutencao e Operacoes	54
12.1 Rotinas de Manutencao	54
12.1.1 Manutencao Diaria	54
12.1.2 Manutencao Mensal	56
12.2 Monitoramento Operacional	58
12.2.1 Dashboard de Status	58
12.2.2 Alertas Automaticos	59
12.3 Procedures de Operacao	61
12.3.1 Gerenciamento de Usuarios	61
12.4 Automacao e Jobs	63
12.4.1 Configuracao de Jobs	63

1 Arquitetura do Banco de Dados

1.1 Arquitetura Geral

O banco de dados FinOps segue uma arquitetura de tres camadas integrada com framework NestJS:

1. **Camada de Aplicacao:** NestJS Framework com TypeORM/Prisma
2. **Camada de Logica:** PostgreSQL com procedures e triggers
3. **Camada de Dados:** Armazenamento fisico com backup automatico

Stack Tecnologica:

- **Backend:** NestJS (Node.js + TypeScript)
- **Banco de Dados:** PostgreSQL 15+
- **ORM:** TypeORM ou Prisma (a definir)
- **Cache:** Redis (opcional para performance)
- **API:** REST/GraphQL endpoints

1.2 Distribuicao dos Dados

1.2.1 Esquemas Logicos

- **public:** Tabelas principais do sistema
- **audit:** Tabelas de auditoria e log
- **config:** Configuracoes e parametros do sistema

1.3 Relacionamentos Principais

1.3.1 Hierarquia de Usuarios

```
1 Usuarios
2 |-- Individual (usuario_id)
3 |-- Grupos (grupo_id)
4 |   |-- Usuario Principal
5 |   |-- Usuario Secundario
6 |   |-- Configuracao do Grupo
```

Listing 1: Estrutura hierarquica de usuarios

1.3.2 Fluxo de Dados Financeiros

1. **Entrada:** Usuario cadastra transacao
2. **Processamento:** Sistema calcula impactos nos orcamentos
3. **Distribuicao:** Dados sao organizados conforme modelo escolhido
4. **Relatorios:** Geracao de dashboards e relatorios

1.4 Estrategia de Particao

Para otimizar performance com grandes volumes de dados:

1.4.1 Particao por Data

- **Transacoes:** Particionadas por mes/ano
- **Logs de Auditoria:** Particionadas por mes
- **Historico:** Dados antigos movidos para tabelas de arquivo

1.4.2 Particao por Usuario

- **Dados Individuais:** Separados por usuario_id
- **Dados de Grupo:** Compartilhados entre membros

1.5 Estrategia de Cache

- **Redis:** Cache de sessao e dados frequentes
- **Materialized Views:** Relatorios pre-calculados
- **Query Cache:** Cache de consultas complexas

1.6 Integracoes Externas

Fase Inicial - Sem APIs Externas:

- **Entrada de Dados:** Manual via interface web/mobile
- **Servicos de Backup:** Backup local e em nuvem
- **Monitoramento:** Logs internos do NestJS

Futuras Integracoes (Opcionais):

- **Open Banking:** Integracao com bancos (futura implementacao)
- **APIs de Cartao:** Sincronizacao automatica (futura implementacao)
- **Servicos de Nuvem:** AWS/GCP para backup avancado

2 Modelo Conceitual

2.1 Diagrama Entidade-Relacionamento

O modelo conceitual representa as entidades principais e seus relacionamentos:

2.1.1 Entidades Principais

Usuario

- Pessoa fisica que utiliza o sistema
- Pode estar associado a um grupo (casal/familia)
- Possui configuracoes individuais

Grupo

- Agrupamento de usuarios (casal, familia, republica)
- Define o modelo de organizacao financeira
- Compartilha dados conforme configuracao

Conta

- Representa contas bancarias, carteiras, investimentos
- Pode ser individual ou compartilhada
- Possui saldo atual e historico

Categoria

- Classificacao de receitas e despesas
- Pode ser personalizada por usuario/grupo
- Possui subcategorias hierarquicas

Transacao

- Movimentacao financeira (entrada ou saida)
- Associada a uma conta e categoria
- Possui informacoes de divisao (para modelo Splitwise)

Orcamento

- Planejamento financeiro por categoria/periodo
- Pode ser individual ou compartilhado
- Possui metas e limites de gasto

Meta

- Objetivos de poupanca ou investimento
- Possui valor alvo e prazo
- Acompanha progresso automaticamente

2.2 Relacionamentos

2.2.1 Usuario - Grupo (N:M)

- Um usuario pode pertencer a multiplos grupos
- Um grupo pode ter multiplos usuarios
- Relacionamento possui papel (principal, secundario, dependente)

2.2.2 Usuario - Conta (1:N)

- Um usuario pode ter multiplas contas
- Uma conta pertence a um usuario especifico
- Contas compartilhadas referenciam o grupo

2.2.3 Conta - Transacao (1:N)

- Uma conta pode ter multiplas transacoes
- Uma transacao pertence a uma conta especifica

2.2.4 Categoria - Transacao (1:N)

- Uma categoria pode ter multiplas transacoes
- Uma transacao possui uma categoria principal

2.2.5 Usuario - Orcamento (1:N)

- Um usuario pode ter multiplos orcamentos
- Um orcamento pode ser individual ou de grupo

2.3 Cardinalidades e Restricoes

- **Usuario:** Obrigatorio ter pelo menos um email valido
- **Grupo:** Minimo 2 usuarios, maximo configuravel
- **Conta:** Deve ter saldo inicial definido
- **Transacao:** Valor deve ser diferente de zero
- **Categoria:** Nome deve ser unico por usuario/grupo

- **Orcamento:** Periodo deve ser valido (inicio < fim)
- **Meta:** Valor alvo deve ser positivo

2.4 Regras de Negocio

2.4.1 Modelos de Organizacao

1. **Individual:** Todas as entidades sao privadas do usuario
2. **Renda Conjunta:** Contas e orcamentos sao compartilhados
3. **Splitwise:** Transacoes individuais, divisao calculada
4. **Fundo Comum:** Hibrido entre individual e compartilhado

2.4.2 Integridade Referencial

- Exclusao de usuario: Transferir dados para outro membro do grupo
- Exclusao de grupo: Converter contas compartilhadas em individuais
- Exclusao de categoria: Mover transacoes para categoria "Outros"
- Exclusao de conta: Transferir saldo para conta principal

3 Modelo Logico

3.1 Estrutura das Tabelas

3.1.1 Tabela: usuarios

```
1 CREATE TABLE usuarios (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     nome VARCHAR(100) NOT NULL,  
4     email VARCHAR(255) UNIQUE NOT NULL,  
5     senha_hash VARCHAR(255) NOT NULL,  
6     telefone VARCHAR(20),  
7     data_nascimento DATE,  
8     cpf VARCHAR(14) UNIQUE,  
9     ativo BOOLEAN DEFAULT true,  
10    email_verificado BOOLEAN DEFAULT false,  
11    data_ultimo_acesso TIMESTAMP,  
12    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
13    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
14 );
```

Listing 2: Estrutura da tabela usuarios

3.1.2 Tabela: grupos

```
1 CREATE TABLE grupos (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     nome VARCHAR(100) NOT NULL,  
4     descricao TEXT,  
5     modelo_organizacao VARCHAR(20) NOT NULL DEFAULT 'individual',  
6     configuracao JSONB,  
7     ativo BOOLEAN DEFAULT true,  
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
9     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10  
11     CONSTRAINT chk_grupos_modelo CHECK (  
12         modelo_organizacao IN ('individual', 'renda_conjunta',  
13         'splitwise', 'fundo_comum')  
14     );
```

Listing 3: Estrutura da tabela grupos

3.1.3 Tabela: usuario_grupos

```
1 CREATE TABLE usuario_grupos (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE  
4     CASCADE,  
5     grupo_id UUID NOT NULL REFERENCES grupos(id) ON DELETE CASCADE,  
6     papel VARCHAR(20) NOT NULL DEFAULT 'membro',  
7     permissoes JSONB,  
8     ativo BOOLEAN DEFAULT true,  
9     data_entrada TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10     data_saida TIMESTAMP,  
11  
12     CONSTRAINT chk_usuario_grupos_papel CHECK (  
13         papel IN ('administrador', 'membro', 'dependente',  
14         'convidado')  
15     ),  
16     UNIQUE($usuario\_id$, $grupo\_id$)  
17 );
```

Listing 4: Estrutura da tabela usuario_grupos(*relacionamento* $N : M$)

3.1.4 Tabela: contas

```
1 CREATE TABLE contas (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     nome VARCHAR(100) NOT NULL,  
4     descricao TEXT,  
5     tipo VARCHAR(30) NOT NULL,  
6     subtipo VARCHAR(30),
```

```
7   moeda VARCHAR(3) DEFAULT 'BRL',
8   saldo_inicial DECIMAL(15,2) DEFAULT 0.00,
9   saldo_atual DECIMAL(15,2) DEFAULT 0.00,
10  limite_credito DECIMAL(15,2),
11  banco VARCHAR(100),
12  agencia VARCHAR(10),
13  numero_conta VARCHAR(20),
14  usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,
15  grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
16  compartilhada BOOLEAN DEFAULT false,
17  ativa BOOLEAN DEFAULT true,
18  cor VARCHAR(7) DEFAULT '#007bff',
19  icone VARCHAR(50),
20  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
21  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
22
23  CONSTRAINT chk_contas_tipo CHECK (
24      tipo IN ('corrente', 'poupanca', 'investimento',
25              'cartao_credito',
26              'cartao_debito', 'dinheiro', 'outros')
27  ),
28  CONSTRAINT chk_contas_saldo CHECK (saldo_atual >= 0 OR tipo =
29      'cartao_credito')
30 );
```

Listing 5: Estrutura da tabela contas

3.1.5 Tabela: categorias

```
1 CREATE TABLE categorias (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     nome VARCHAR(100) NOT NULL,
4     descricao TEXT,
5     tipo VARCHAR(20) NOT NULL,
6     cor VARCHAR(7) DEFAULT '#6c757d',
7     icone VARCHAR(50),
8     categoria_pai_id UUID REFERENCES categorias(id) ON DELETE SET
9     NULL,
10    usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,
11    grupo_id UUID REFERENCES grupos(id) ON DELETE CASCADE,
12    sistema BOOLEAN DEFAULT false,
13    ativa BOOLEAN DEFAULT true,
14    ordem INTEGER DEFAULT 0,
15    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
16    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
17
18    CONSTRAINT chk_categorias_tipo CHECK (
19        tipo IN ('receita', 'despesa', 'transferencia')
20    ),
21    CONSTRAINT chk_categorias_escopo CHECK (
22        (usuario_id IS NOT NULL AND grupo_id IS NULL) OR
```

```
22         (usuario_id IS NULL AND grupo_id IS NOT NULL) OR
23         (usuario_id IS NULL AND grupo_id IS NULL AND sistema =
24         true)
25     );
```

Listing 6: Estrutura da tabela categorias

3.1.6 Tabela: transacoes

```
1 CREATE TABLE transacoes (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     descricao VARCHAR(255) NOT NULL,
4     valor DECIMAL(15,2) NOT NULL,
5     tipo VARCHAR(20) NOT NULL,
6     data_transacao DATE NOT NULL,
7     data_vencimento DATE,
8     conta_id UUID NOT NULL REFERENCES contas(id) ON DELETE
9     RESTRICT,
10    categoria_id UUID NOT NULL REFERENCES categorias(id) ON DELETE
11    RESTRICT,
12    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE
13    CASCADE,
14    grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
15    conta_destino_id UUID REFERENCES contas(id) ON DELETE SET NULL,
16    status VARCHAR(20) DEFAULT 'confirmada',
17    observacoes TEXT,
18    tags TEXT[],
19    anexos JSONB,
20    recorrencia JSONB,
21    divisao_splitwise JSONB,
22    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
23    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
24
25    CONSTRAINT chk_transacoes_tipo CHECK (
26        tipo IN ('receita', 'despesa', 'transferencia')
27    ),
28    CONSTRAINT chk_transacoes_status CHECK (
29        status IN ('pendente', 'confirmada', 'cancelada',
30        'agendada')
31    ),
32    CONSTRAINT chk_transacoes_valor CHECK (valor != 0),
33    CONSTRAINT chk_transacoes_transferencia CHECK (
34        (tipo = 'transferencia' AND conta_destino_id IS NOT NULL)
35    OR
36        (tipo != 'transferencia' AND conta_destino_id IS NULL)
37    )
38 );
```

Listing 7: Estrutura da tabela transacoes

3.2 Indices Principais

```
1 -- Indices para consultas frequentes
2 CREATE INDEX idx_usuarios_email ON usuarios(email);
3 CREATE INDEX idx_usuarios_ativo ON usuarios(ativo);
4
5 CREATE INDEX idx_contas_usuario ON contas(usuario_id);
6 CREATE INDEX idx_contas_grupo ON contas(grupo_id);
7 CREATE INDEX idx_contas_ativa ON contas(ativa);
8
9 CREATE INDEX idx_transacoes_conta ON transacoes(conta_id);
10 CREATE INDEX idx_transacoes_categoria ON transacoes(categoria_id);
11 CREATE INDEX idx_transacoes_usuario ON transacoes(usuario_id);
12 CREATE INDEX idx_transacoes_data ON transacoes(data_transacao);
13 CREATE INDEX idx_transacoes_status ON transacoes(status);
14
15 -- Indices compostos para consultas complexas
16 CREATE INDEX idx_transacoes_usuario_data ON transacoes(usuario_id,
17     data_transacao DESC);
18 CREATE INDEX idx_transacoes_conta_data ON transacoes(conta_id,
19     data_transacao DESC);
20 CREATE INDEX idx_contas_usuario_ativa ON contas(usuario_id, ativa);
```

Listing 8: Indices para otimizacao de performance

3.3 Constraints e Validacoes

```
1 -- Validacao de email
2 ALTER TABLE usuarios ADD CONSTRAINT chk_usuarios_email
3     CHECK (email ~*
4         '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');
5
6 -- Validacao de CPF (formato)
7 ALTER TABLE usuarios ADD CONSTRAINT chk_usuarios_cpf
8     CHECK (cpf IS NULL OR cpf ~ '^\d{3}\.\d{3}\.\d{3}-\d{2}$');
9
10 -- Validacao de saldo para cartao de credito
11 ALTER TABLE contas ADD CONSTRAINT chk_contas_limite_credito
12     CHECK (limite_credito IS NULL OR limite_credito >= 0);
13
14 -- Validacao de hierarquia de categorias (evitar loop)
15 CREATE OR REPLACE FUNCTION check_categoria_hierarchy() RETURNS
16     TRIGGER AS $$
17 BEGIN
18     IF NEW.categoria_pai_id IS NOT NULL AND
19     NEW.categoria_pai_id = NEW.id THEN
20         RAISE EXCEPTION 'Categoria nao pode ser pai de si mesma';
21     END IF;
22     RETURN NEW;
23 END;
24 $$ LANGUAGE plpgsql;
```

```
23  
24 CREATE TRIGGER trg_check_categoria_hierarchy  
25 BEFORE INSERT OR UPDATE ON categorias  
26 FOR EACH ROW EXECUTE FUNCTION check_categoria_hierarchy();
```

Listing 9: Constraints adicionais de integridade

4 Modelo Fisico

4.1 Implementacao Completa

4.1.1 Tabelas Complementares

Tabela: orcamentos

```
1 CREATE TABLE orcamentos (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     nome VARCHAR(100) NOT NULL,  
4     descricao TEXT,  
5     valor_planejado DECIMAL(15,2) NOT NULL,  
6     valor_gasto DECIMAL(15,2) DEFAULT 0.00,  
7     periodo_inicio DATE NOT NULL,  
8     periodo_fim DATE NOT NULL,  
9     categoria_id UUID REFERENCES categorias(id) ON DELETE CASCADE,  
10    usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,  
11    grupo_id UUID REFERENCES grupos(id) ON DELETE CASCADE,  
12    tipo_periodo VARCHAR(20) DEFAULT 'mensal',  
13    alerta_percentual INTEGER DEFAULT 80,  
14    ativo BOOLEAN DEFAULT true,  
15    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
16    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
17  
18    CONSTRAINT chk_orcamentos_periodo CHECK (periodo_inicio <=  
19    periodo_fim),  
20    CONSTRAINT chk_orcamentos_valor CHECK (valor_planejado > 0),  
21    CONSTRAINT chk_orcamentos_alerta CHECK (alerta_percentual  
22    BETWEEN 0 AND 100),  
23    CONSTRAINT chk_orcamentos_escopo CHECK (  
24        (usuario_id IS NOT NULL AND grupo_id IS NULL) OR  
25        (usuario_id IS NULL AND grupo_id IS NOT NULL)  
26    )  
27 );
```

Listing 10: Estrutura da tabela orcamentos

Tabela: metas

```
1 CREATE TABLE metas (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     nome VARCHAR(100) NOT NULL,  
4     descricao TEXT,  
5     valor_alvo DECIMAL(15,2) NOT NULL,  
6     valor_atual DECIMAL(15,2) DEFAULT 0.00,
```

```

7      data_alvo DATE,
8      tipo VARCHAR(30) NOT NULL,
9      categoria_id UUID REFERENCES categorias(id) ON DELETE SET NULL,
10     conta_destino_id UUID REFERENCES contas(id) ON DELETE SET NULL,
11     usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,
12     grupo_id UUID REFERENCES grupos(id) ON DELETE CASCADE,
13     frequencia_contribuicao VARCHAR(20),
14     valor_contribuicao DECIMAL(15,2),
15     ativa BOOLEAN DEFAULT true,
16     alcancada BOOLEAN DEFAULT false,
17     data_alcancada TIMESTAMP,
18     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
19     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
20
21     CONSTRAINT chk_metas_valor CHECK (valor_alvo > 0),
22     CONSTRAINT chk_metas_tipo CHECK (
23         tipo IN ('poupanca', 'investimento', 'emergencia',
24 'compra', 'viagem', 'outros')
25     ),
26     CONSTRAINT chk_metas_frequencia CHECK (
27         frequencia_contribuicao IS NULL OR
28         frequencia_contribuicao IN ('diario', 'semanal', 'mensal',
29 'anual')
30     ),
31     CONSTRAINT chk_metas_escopo CHECK (
32         (usuario_id IS NOT NULL AND grupo_id IS NULL) OR
33         (usuario_id IS NULL AND grupo_id IS NOT NULL)
34     );

```

Listing 11: Estrutura da tabela metas

Tabela: recorrências

```

1 CREATE TABLE recorrências (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     nome VARCHAR(100) NOT NULL,
4     descricao TEXT,
5     valor DECIMAL(15,2) NOT NULL,
6     tipo VARCHAR(20) NOT NULL,
7     frequencia VARCHAR(20) NOT NULL,
8     dia_vencimento INTEGER,
9     data_inicio DATE NOT NULL,
10    data_fim DATE,
11    conta_id UUID NOT NULL REFERENCES contas(id) ON DELETE CASCADE,
12    categoria_id UUID NOT NULL REFERENCES categorias(id) ON DELETE
13    RESTRICT,
14    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE
15    CASCADE,
16    grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
17    proxima_execucao DATE NOT NULL,
18    ativa BOOLEAN DEFAULT true,
19    auto_confirmar BOOLEAN DEFAULT false,

```

```

18 observacoes TEXT,
19 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
20 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
21
22 CONSTRAINT chk_recorrencias_tipo CHECK (
23     tipo IN ('receita', 'despesa')
24 ),
25 CONSTRAINT chk_recorrencias_frequencia CHECK (
26     frequencia IN ('diario', 'semanal', 'quinzenal', 'mensal',
27 'bimestral',
28 'trimestral', 'semestral', 'anual')
29 ),
30 CONSTRAINT chk_recorrencias_dia CHECK (
31     dia_vencimento IS NULL OR dia_vencimento BETWEEN 1 AND 31
32 ),
33 CONSTRAINT chk_recorrencias_datas CHECK (
34     data_fim IS NULL OR data_fim >= data_inicio
35 );

```

Listing 12: Estrutura da tabela recorrencias

Tabela: splitwise_divisoas

```

1 CREATE TABLE splitwise_divisoas (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     transacao_id UUID NOT NULL REFERENCES transacoes(id) ON DELETE
4     CASCADE,
5     usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE
6     CASCADE,
7     valor_original DECIMAL(15,2) NOT NULL,
8     percentual_divisao DECIMAL(5,2) DEFAULT 50.00,
9     valor_devido DECIMAL(15,2) NOT NULL,
10    pago BOOLEAN DEFAULT false,
11    data_pagamento TIMESTAMP,
12    observacoes TEXT,
13    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
14    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
15
16    CONSTRAINT chk_splitwise_percentual CHECK (
17        percentual_divisao >= 0 AND percentual_divisao <= 100
18    ),
19    UNIQUE($transacao\_id$, $usuario\_id$)
20 );

```

Listing 13: Estrutura da tabela splitwise

divisoas

4.1.2 Tabelas de Auditoria

Tabela: audit_logs

```

1 CREATE TABLE audit.audit_logs (
2     id BIGSERIAL PRIMARY KEY,
3     tabela VARCHAR(100) NOT NULL,

```



```

4      operacao VARCHAR(10) NOT NULL,
5      usuario_id UUID,
6      dados_antigos JSONB,
7      dados_novos JSONB,
8      timestamp_operacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9      ip_origem INET,
10     user_agent TEXT,
11
12     CONSTRAINT chk_audit_operacao CHECK (
13         operacao IN ('INSERT', 'UPDATE', 'DELETE')
14     )
15 ) PARTITION BY RANGE ($timestamp\_operacao$);

```

Listing 14: Estrutura da tabela *audit_logs***Tabela: sessoes**

```

1 CREATE TABLE sessoes (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE
4     CASCADE,
5     token_hash VARCHAR(255) NOT NULL,
6     ip_origem INET NOT NULL,
7     user_agent TEXT,
8     data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     data_expiracao TIMESTAMP NOT NULL,
10    data_ultimo_acesso TIMESTAMP,
11    ativa BOOLEAN DEFAULT true,
12
13    CONSTRAINT chk_sesoes_expiracao CHECK (data_expiracao >
14    data_criacao)
15 );

```

Listing 15: Estrutura da tabela sessoes

4.2 Views Materializadas

View: resumo_financeiro_mensal

```

1 CREATE MATERIALIZED VIEW resumo_financeiro_mensal AS
2 SELECT
3     usuario_id,
4     grupo_id,
5     DATE_TRUNC('month', data_transacao) AS mes_ano,
6     SUM(CASE WHEN tipo = 'receita' THEN valor ELSE 0 END) AS
7     total_receitas,
8     SUM(CASE WHEN tipo = 'despesa' THEN valor ELSE 0 END) AS
9     total_despesas,
10    SUM(CASE WHEN tipo = 'receita' THEN valor ELSE -valor END) AS
11    saldo_mensal,
12    COUNT(*) AS total_transacoes
13 FROM transacoes
14 WHERE status = 'confirmada'
15 GROUP BY usuario_id, grupo_id, DATE_TRUNC('month', data_transacao);

```

```
13
14 -- Indice para performance
15 CREATE UNIQUE INDEX idx_resumo_financeiro_pk
16 ON resumo_financeiro_mensal(usuario_id, COALESCE(grupo_id,
'00000000-0000-0000-0000-000000000000'::uuid), mes_ano);
```

Listing 16: View materializada para resumos mensais

4.3 Funcoes de Sistema

Funcao: atualizar_saldo_conta

```
1 CREATE OR REPLACE FUNCTION atualizar_saldo_conta(conta_uuid UUID)
2 RETURNS DECIMAL(15,2) AS $$
3 DECLARE
4     novo_saldo DECIMAL(15,2);
5     saldo_inicial DECIMAL(15,2);
6 BEGIN
7     -- Buscar saldo inicial
8     SELECT c.saldo_inicial INTO saldo_inicial
9     FROM contas c WHERE c.id = conta_uuid;
10
11     -- Calcular novo saldo
12     SELECT saldo_inicial + COALESCE(SUM(
13         CASE
14             WHEN t.tipo = 'receita' THEN t.valor
15             WHEN t.tipo = 'despesa' THEN -t.valor
16             WHEN t.tipo = 'transferencia' AND t.conta_id =
17 conta_uuid THEN -t.valor
18             WHEN t.tipo = 'transferencia' AND t.conta_destino_id =
19 conta_uuid THEN t.valor
20             ELSE 0
21         END
22     ), 0) INTO novo_saldo
23     FROM transacoes t
24     WHERE (t.conta_id = conta_uuid OR t.conta_destino_id =
25 conta_uuid)
26     AND t.status = 'confirmada';
27
28     -- Atualizar saldo na conta
29     UPDATE contas SET
30         saldo_atual = novo_saldo,
31         updated_at = CURRENT_TIMESTAMP
32     WHERE id = conta_uuid;
33
34     RETURN novo_saldo;
35 END;
36 $$ LANGUAGE plpgsql;
```

Listing 17: Funcao para atualizar saldo das contas

5 Scripts de Criacao

5.1 Script Principal de Instalacao

```
1  -- =====
2  -- FINOPS DATABASE CREATION SCRIPT
3  -- Versao: 1.0
4  -- Data: \today
5  -- =====
6
7  -- Criar extensoes necessarias
8  CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
9  CREATE EXTENSION IF NOT EXISTS "pg_stat_statements";
10
11 -- Criar esquemas
12 CREATE SCHEMA IF NOT EXISTS audit;
13 CREATE SCHEMA IF NOT EXISTS config;
14
15 -- =====
16 -- TABELAS PRINCIPAIS
17 -- =====
18
19 -- Usuarios
20 CREATE TABLE usuarios (
21     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
22     nome VARCHAR(100) NOT NULL,
23     email VARCHAR(255) UNIQUE NOT NULL,
24     senha_hash VARCHAR(255) NOT NULL,
25     telefone VARCHAR(20),
26     data_nascimento DATE,
27     cpf VARCHAR(14) UNIQUE,
28     ativo BOOLEAN DEFAULT true,
29     email_verificado BOOLEAN DEFAULT false,
30     data_ultimo_acesso TIMESTAMP,
31     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
33 );
34
35 -- Grupos
36 CREATE TABLE grupos (
37     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
38     nome VARCHAR(100) NOT NULL,
39     descricao TEXT,
40     modelo_organizacao VARCHAR(20) NOT NULL DEFAULT 'individual',
41     configuracao JSONB,
42     ativo BOOLEAN DEFAULT true,
43     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
44     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
45 );
46
47 -- Usuario-Grupos (relacionamento N:M)
```

```
48 CREATE TABLE usuario_grupos (  
49     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
50     usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE  
    CASCADE,  
51     grupo_id UUID NOT NULL REFERENCES grupos(id) ON DELETE CASCADE,  
52     papel VARCHAR(20) NOT NULL DEFAULT 'membro',  
53     permissoes JSONB,  
54     ativo BOOLEAN DEFAULT true,  
55     data_entrada TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
56     data_saida TIMESTAMP,  
57     UNIQUE(usuario_id, grupo_id)  
58 );  
59  
60 -- Categorias  
61 CREATE TABLE categorias (  
62     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
63     nome VARCHAR(100) NOT NULL,  
64     descricao TEXT,  
65     tipo VARCHAR(20) NOT NULL,  
66     cor VARCHAR(7) DEFAULT '#6c757d',  
67     icone VARCHAR(50),  
68     categoria_pai_id UUID REFERENCES categorias(id) ON DELETE SET  
    NULL,  
69     usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,  
70     grupo_id UUID REFERENCES grupos(id) ON DELETE CASCADE,  
71     sistema BOOLEAN DEFAULT false,  
72     ativa BOOLEAN DEFAULT true,  
73     ordem INTEGER DEFAULT 0,  
74     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
75     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
76 );  
77  
78 -- Contas  
79 CREATE TABLE contas (  
80     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
81     nome VARCHAR(100) NOT NULL,  
82     descricao TEXT,  
83     tipo VARCHAR(30) NOT NULL,  
84     subtipo VARCHAR(30),  
85     moeda VARCHAR(3) DEFAULT 'BRL',  
86     saldo_inicial DECIMAL(15,2) DEFAULT 0.00,  
87     saldo_atual DECIMAL(15,2) DEFAULT 0.00,  
88     limite_credito DECIMAL(15,2),  
89     banco VARCHAR(100),  
90     agencia VARCHAR(10),  
91     numero_conta VARCHAR(20),  
92     usuario_id UUID REFERENCES usuarios(id) ON DELETE CASCADE,  
93     grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,  
94     compartilhada BOOLEAN DEFAULT false,  
95     ativa BOOLEAN DEFAULT true,  
96     cor VARCHAR(7) DEFAULT '#007bff',
```

```

97     icone VARCHAR(50),
98     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
99     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
100 );
101
102 -- Transacoes
103 CREATE TABLE transacoes (
104     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
105     descricao VARCHAR(255) NOT NULL,
106     valor DECIMAL(15,2) NOT NULL,
107     tipo VARCHAR(20) NOT NULL,
108     data_transacao DATE NOT NULL,
109     data_vencimento DATE,
110     conta_id UUID NOT NULL REFERENCES contas(id) ON DELETE
111     RESTRICT,
112     categoria_id UUID NOT NULL REFERENCES categorias(id) ON DELETE
113     RESTRICT,
114     usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE
115     CASCADE,
116     grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
117     conta_destino_id UUID REFERENCES contas(id) ON DELETE SET NULL,
118     status VARCHAR(20) DEFAULT 'confirmada',
119     observacoes TEXT,
120     tags TEXT[],
121     anexos JSONB,
122     recorrencia JSONB,
123     divisao_splitwise JSONB,
124     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
125     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
126 );
127
128 -- =====
129 -- CONSTRAINTS
130 -- =====
131
132 -- Usuarios
133 ALTER TABLE usuarios ADD CONSTRAINT chk_usuarios_email
134     CHECK (email ~*
135         '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');
136 ALTER TABLE usuarios ADD CONSTRAINT chk_usuarios_cpf
137     CHECK (cpf IS NULL OR cpf ~ '^\d{3}\.\d{3}\.\d{3}-\d{2}$');
138
139 -- Grupos
140 ALTER TABLE grupos ADD CONSTRAINT chk_grupos_modelo
141     CHECK (modelo_organizacao IN ('individual', 'renda_conjunta',
142     'splitwise', 'fundo_comum'));
143
144 -- Usuario-Grupos
145 ALTER TABLE usuario_grupos ADD CONSTRAINT chk_usuario_grupos_papel
146     CHECK (papel IN ('administrador', 'membro', 'dependente',
147     'convidado'));

```

```
142
143 -- Categorias
144 ALTER TABLE categorias ADD CONSTRAINT chk_categorias_tipo
145     CHECK (tipo IN ('receita', 'despesa', 'transferencia'));
146 ALTER TABLE categorias ADD CONSTRAINT chk_categorias_escopo
147     CHECK ((usuario_id IS NOT NULL AND grupo_id IS NULL) OR
148           (usuario_id IS NULL AND grupo_id IS NOT NULL) OR
149           (usuario_id IS NULL AND grupo_id IS NULL AND sistema =
150             true));
151
152 -- Contas
153 ALTER TABLE contas ADD CONSTRAINT chk_contas_tipo
154     CHECK (tipo IN ('corrente', 'poupanca', 'investimento',
155                    'cartao_credito',
156                    'cartao_debito', 'dinheiro', 'outros'));
157 ALTER TABLE contas ADD CONSTRAINT chk_contas_saldo
158     CHECK (saldo_atual >= 0 OR tipo = 'cartao_credito');
159
160 -- Transacoes
161 ALTER TABLE transacoes ADD CONSTRAINT chk_transacoes_tipo
162     CHECK (tipo IN ('receita', 'despesa', 'transferencia'));
163 ALTER TABLE transacoes ADD CONSTRAINT chk_transacoes_status
164     CHECK (status IN ('pendente', 'confirmada', 'cancelada',
165                      'agendada'));
166 ALTER TABLE transacoes ADD CONSTRAINT chk_transacoes_valor
167     CHECK (valor != 0);
168 ALTER TABLE transacoes ADD CONSTRAINT chk_transacoes_transferencia
169     CHECK ((tipo = 'transferencia' AND conta_destino_id IS NOT
170            NULL) OR
171           (tipo != 'transferencia' AND conta_destino_id IS NULL));
172
173 -- =====
174 -- INDICES
175 -- =====
176
177 -- Usuarios
178 CREATE INDEX idx_usuarios_email ON usuarios(email);
179 CREATE INDEX idx_usuarios_ativo ON usuarios(ativo);
180 CREATE INDEX idx_usuarios_cpf ON usuarios(cpf) WHERE cpf IS NOT
181     NULL;
182
183 -- Grupos
184 CREATE INDEX idx_grupos_ativo ON grupos(ativo);
185 CREATE INDEX idx_grupos_modelo ON grupos(modelo_organizacao);
186
187 -- Usuario-Grupos
188 CREATE INDEX idx_usuario_grupos_usuario ON
189     usuario_grupos(usuario_id);
190 CREATE INDEX idx_usuario_grupos_grupo ON usuario_grupos(grupo_id);
191 CREATE INDEX idx_usuario_grupos_ativo ON usuario_grupos(ativo);
```

```
187 -- Categorias
188 CREATE INDEX idx_categorias_usuario ON categorias(usuario_id);
189 CREATE INDEX idx_categorias_grupo ON categorias(grupo_id);
190 CREATE INDEX idx_categorias_tipo ON categorias(tipo);
191 CREATE INDEX idx_categorias_pai ON categorias(categoria_pai_id);
192 CREATE INDEX idx_categorias_ativa ON categorias(ativa);
193
194 -- Contas
195 CREATE INDEX idx_contas_usuario ON contas(usuario_id);
196 CREATE INDEX idx_contas_grupo ON contas(grupo_id);
197 CREATE INDEX idx_contas_tipo ON contas(tipo);
198 CREATE INDEX idx_contas_ativa ON contas(ativa);
199 CREATE INDEX idx_contas_usuario_ativa ON contas(usuario_id, ativa);
200
201 -- Transacoes
202 CREATE INDEX idx_transacoes_conta ON transacoes(conta_id);
203 CREATE INDEX idx_transacoes_categoria ON transacoes(categoria_id);
204 CREATE INDEX idx_transacoes_usuario ON transacoes(usuario_id);
205 CREATE INDEX idx_transacoes_grupo ON transacoes(grupo_id);
206 CREATE INDEX idx_transacoes_data ON transacoes(data_transacao);
207 CREATE INDEX idx_transacoes_status ON transacoes(status);
208 CREATE INDEX idx_transacoes_tipo ON transacoes(tipo);
209 CREATE INDEX idx_transacoes_usuario_data ON transacoes(usuario_id,
    data_transacao DESC);
210 CREATE INDEX idx_transacoes_conta_data ON transacoes(conta_id,
    data_transacao DESC);
211
212 -- =====
213 -- DADOS INICIAIS
214 -- =====
215
216 -- Categorias do sistema
217 INSERT INTO categorias (nome, tipo, cor, icone, sistema) VALUES
218     ('Salario', 'receita', '#28a745', 'money', true),
219     ('Freelance', 'receita', '#17a2b8', 'briefcase', true),
220     ('Investimentos', 'receita', '#6f42c1', 'trending-up', true),
221     ('Outros Rendimentos', 'receita', '#6c757d', 'plus-circle',
    true),
222
223     ('Alimentacao', 'despesa', '#fd7e14', 'utensils', true),
224     ('Transporte', 'despesa', '#20c997', 'car', true),
225     ('Moradia', 'despesa', '#dc3545', 'home', true),
226     ('Saude', 'despesa', '#e83e8c', 'heart', true),
227     ('Educacao', 'despesa', '#6f42c1', 'book', true),
228     ('Lazer', 'despesa', '#ffc107', 'smile', true),
229     ('Roupas', 'despesa', '#17a2b8', 'shopping-bag', true),
230     ('Outros Gastos', 'despesa', '#6c757d', 'more-horizontal',
    true),
231
232     ('Transferencia', 'transferencia', '#007bff', 'arrow-right',
    true);
```

```
233  
234 COMMIT;
```

Listing 18: Script completo de criacao do banco

5.2 Script de Rollback

```
1  -- =====  
2  -- FINOPS DATABASE ROLLBACK SCRIPT  
3  -- =====  
4  
5  -- Remover indices  
6  DROP INDEX IF EXISTS idx_transacoes_conta_data;  
7  DROP INDEX IF EXISTS idx_transacoes_usuario_data;  
8  DROP INDEX IF EXISTS idx_transacoes_tipo;  
9  DROP INDEX IF EXISTS idx_transacoes_status;  
10 DROP INDEX IF EXISTS idx_transacoes_data;  
11 DROP INDEX IF EXISTS idx_transacoes_grupo;  
12 DROP INDEX IF EXISTS idx_transacoes_usuario;  
13 DROP INDEX IF EXISTS idx_transacoes_categoria;  
14 DROP INDEX IF EXISTS idx_transacoes_conta;  
15  
16 DROP INDEX IF EXISTS idx_contas_usuario_ativa;  
17 DROP INDEX IF EXISTS idx_contas_ativa;  
18 DROP INDEX IF EXISTS idx_contas_tipo;  
19 DROP INDEX IF EXISTS idx_contas_grupo;  
20 DROP INDEX IF EXISTS idx_contas_usuario;  
21  
22 DROP INDEX IF EXISTS idx_categorias_ativa;  
23 DROP INDEX IF EXISTS idx_categorias_pai;  
24 DROP INDEX IF EXISTS idx_categorias_tipo;  
25 DROP INDEX IF EXISTS idx_categorias_grupo;  
26 DROP INDEX IF EXISTS idx_categorias_usuario;  
27  
28 DROP INDEX IF EXISTS idx_usuario_grupos_ativo;  
29 DROP INDEX IF EXISTS idx_usuario_grupos_grupo;  
30 DROP INDEX IF EXISTS idx_usuario_grupos_usuario;  
31  
32 DROP INDEX IF EXISTS idx_grupos_modelo;  
33 DROP INDEX IF EXISTS idx_grupos_ativo;  
34  
35 DROP INDEX IF EXISTS idx_usuarios_cpf;  
36 DROP INDEX IF EXISTS idx_usuarios_ativo;  
37 DROP INDEX IF EXISTS idx_usuarios_email;  
38  
39 -- Remover tabelas (ordem importa devido aos relacionamentos)  
40 DROP TABLE IF EXISTS transacoes CASCADE;  
41 DROP TABLE IF EXISTS contas CASCADE;  
42 DROP TABLE IF EXISTS categorias CASCADE;  
43 DROP TABLE IF EXISTS usuario_grupos CASCADE;  
44 DROP TABLE IF EXISTS grupos CASCADE;
```



```
45 DROP TABLE IF EXISTS usuarios CASCADE;
46
47 -- Remover esquemas
48 DROP SCHEMA IF EXISTS config CASCADE;
49 DROP SCHEMA IF EXISTS audit CASCADE;
50
51 -- Remover extensoes (opcional - pode afetar outros bancos)
52 -- DROP EXTENSION IF EXISTS "pg_stat_statements";
53 -- DROP EXTENSION IF EXISTS "uuid-oss";
```

Listing 19: Script para reverter instalacao

6 Procedures e Funcoes

6.1 Funcoes de Negocio

6.1.1 Calculo Splitwise

```
1 CREATE OR REPLACE FUNCTION calcular_splitwise_grupo(
2     p_grupo_id UUID,
3     p_data_inicio DATE DEFAULT DATE_TRUNC('month',
4     CURRENT_DATE)::DATE,
5     p_data_fim DATE DEFAULT (DATE_TRUNC('month', CURRENT_DATE) +
6     INTERVAL '1 month - 1 day')::DATE
7 )
8 RETURNS TABLE (
9     usuario_id UUID,
10    nome_usuario VARCHAR(100),
11    total_pago DECIMAL(15,2),
12    valor_devido DECIMAL(15,2),
13    saldo DECIMAL(15,2)
14 ) AS $$
15 DECLARE
16     total_gastos DECIMAL(15,2);
17     num_usuarios INTEGER;
18     valor_por_pessoa DECIMAL(15,2);
19 BEGIN
20     -- Verificar se o grupo usa modelo Splitwise
21     IF NOT EXISTS (
22         SELECT 1 FROM grupos
23         WHERE id = p_grupo_id AND modelo_organizacao = 'splitwise'
24     ) THEN
25         RAISE EXCEPTION 'Grupo nao configurado para modelo
26 Splitwise';
27     END IF;
28
29     -- Contar usuarios ativos no grupo
30     SELECT COUNT(*) INTO num_usuarios
31     FROM usuario_grupos ug
32     WHERE ug.grupo_id = p_grupo_id AND ug.ativo = true;
```

```

30
31  -- Calcular total de gastos do periodo
32  SELECT COALESCE(SUM(t.valor), 0) INTO total_gastos
33  FROM transacoes t
34  WHERE t.grupo_id = p_grupo_id
35         AND t.tipo = 'despesa'
36         AND t.status = 'confirmada'
37         AND t.data_transacao BETWEEN p_data_inicio AND p_data_fim;
38
39  -- Calcular valor devido por pessoa
40  valor_por_pessoa := total_gastos / num_usuarios;
41
42  -- Retornar resultado
43  RETURN QUERY
44  SELECT
45      u.id,
46      u.nome,
47      COALESCE(SUM(t.valor), 0) AS total_pago,
48      valor_por_pessoa AS valor_devido,
49      (valor_por_pessoa - COALESCE(SUM(t.valor), 0)) AS saldo
50  FROM usuarios u
51  JOIN usuario_grupos ug ON u.id = ug.usuario_id
52  LEFT JOIN transacoes t ON u.id = t.usuario_id
53         AND t.grupo_id = p_grupo_id
54         AND t.tipo = 'despesa'
55         AND t.status = 'confirmada'
56         AND t.data_transacao BETWEEN p_data_inicio AND p_data_fim
57  WHERE ug.grupo_id = p_grupo_id AND ug.ativo = true
58  GROUP BY u.id, u.nome, valor_por_pessoa
59  ORDER BY u.nome;
60 END;
61 $$ LANGUAGE plpgsql;

```

Listing 20: Funcao para calcular divisao Splitwise

6.1.2 Atualizacao de Saldos

```

1 CREATE OR REPLACE PROCEDURE atualizar_saldos_usuario(p_usuario_id
2   UUID)
3 LANGUAGE plpgsql AS $$
4 DECLARE
5     conta_record RECORD;
6     novo_saldo DECIMAL(15,2);
7 BEGIN
8     -- Percorrer todas as contas do usuario
9     FOR conta_record IN
10         SELECT id, saldo_inicial, tipo
11         FROM contas
12         WHERE usuario_id = p_usuario_id AND ativa = true
13     LOOP
14         -- Calcular novo saldo

```

```
14      SELECT conta_record.saldo_inicial + COALESCE(SUM(  
15          CASE  
16              WHEN t.tipo = 'receita' THEN t.valor  
17              WHEN t.tipo = 'despesa' THEN -t.valor  
18              WHEN t.tipo = 'transferencia' AND t.conta_id =  
conta_record.id THEN -t.valor  
19              WHEN t.tipo = 'transferencia' AND  
t.conta_destino_id = conta_record.id THEN t.valor  
20              ELSE 0  
21          END  
22      ), 0) INTO novo_saldo  
23      FROM transacoes t  
24      WHERE (t.conta_id = conta_record.id OR t.conta_destino_id  
= conta_record.id)  
25          AND t.status = 'confirmada';  
26  
27      -- Atualizar saldo  
28      UPDATE contas  
29      SET saldo_atual = novo_saldo,  
30          updated_at = CURRENT_TIMESTAMP  
31      WHERE id = conta_record.id;  
32  
33      -- Log da atualizacao  
34      INSERT INTO audit.audit_logs (tabela, operacao,  
usuario_id, dados_novos)  
35      VALUES ('contas', 'UPDATE', p_usuario_id,  
36          jsonb_build_object('conta_id', conta_record.id,  
'novo_saldo', novo_saldo));  
37      END LOOP;  
38  
39      COMMIT;  
40  END;  
41  $$;
```

Listing 21: Procedure para atualizar saldos de todas as contas

6.1.3 Processamento de Recorrências

```
1  CREATE OR REPLACE FUNCTION processar_recorrencias()  
2  RETURNS INTEGER AS $$  
3  DECLARE  
4      rec_record RECORD;  
5      nova_transacao_id UUID;  
6      contador INTEGER := 0;  
7      proxima_data DATE;  
8  BEGIN  
9      -- Processar recorrências vencidas  
10     FOR rec_record IN  
11         SELECT * FROM recorrências  
12         WHERE ativa = true  
13         AND proxima_execucao <= CURRENT_DATE
```

```
14         AND (data_fim IS NULL OR proxima_execucao <= data_fim)
15     LOOP
16         -- Criar nova transacao
17         INSERT INTO transacoes (
18             descricao, valor, tipo, data_transacao,
19             conta_id, categoria_id, usuario_id, grupo_id,
20             status, observacoes
21         ) VALUES (
22             rec_record.nome,
23             rec_record.valor,
24             rec_record.tipo,
25             rec_record.proxima_execucao,
26             rec_record.conta_id,
27             rec_record.categoria_id,
28             rec_record.usuario_id,
29             rec_record.grupo_id,
30             CASE WHEN rec_record.auto_confirmar THEN 'confirmada'
ELSE 'pendente' END,
31             'Transacao recorrente gerada automaticamente'
32         ) RETURNING id INTO nova_transacao_id;
33
34         -- Calcular proxima execucao
35         SELECT CASE rec_record.frequencia
36             WHEN 'diario' THEN rec_record.proxima_execucao +
INTERVAL '1 day'
37             WHEN 'semanal' THEN rec_record.proxima_execucao +
INTERVAL '1 week'
38             WHEN 'quinzenal' THEN rec_record.proxima_execucao +
INTERVAL '15 days'
39             WHEN 'mensal' THEN rec_record.proxima_execucao +
INTERVAL '1 month'
40             WHEN 'bimestral' THEN rec_record.proxima_execucao +
INTERVAL '2 months'
41             WHEN 'trimestral' THEN rec_record.proxima_execucao +
INTERVAL '3 months'
42             WHEN 'semestral' THEN rec_record.proxima_execucao +
INTERVAL '6 months'
43             WHEN 'anual' THEN rec_record.proxima_execucao +
INTERVAL '1 year'
44             ELSE rec_record.proxima_execucao + INTERVAL '1 month'
45         END INTO proxima_data;
46
47         -- Atualizar recorrencia
48         UPDATE recorrencias
49         SET proxima_execucao = proxima_data,
50             updated_at = CURRENT_TIMESTAMP
51         WHERE id = rec_record.id;
52
53         contador := contador + 1;
54
55         -- Log da operacao
```

```
56      INSERT INTO audit.audit_logs (tabela, operacao,
57      usuario_id, dados_novos)
58      VALUES ('recorrencias', 'PROCESS', rec_record.usuario_id,
59              jsonb_build_object('recorrencia_id', rec_record.id,
60                                'transacao_id', nova_transacao_id,
61                                'proxima_data', proxima_data));
62
63      END LOOP;
64
65      RETURN contador;
66 END;
67 $$ LANGUAGE plpgsql;
```

Listing 22: Funcao para processar transacoes recorrentes

6.2 Funcoes de Relatorio

6.2.1 Resumo Financeiro

```
1 CREATE OR REPLACE FUNCTION gerar_resumo_financeiro(
2     p_usuario_id UUID,
3     p_data_inicio DATE,
4     p_data_fim DATE
5 )
6 RETURNS TABLE (
7     periodo TEXT,
8     total_receitas DECIMAL(15,2),
9     total_despesas DECIMAL(15,2),
10    saldo_periodo DECIMAL(15,2),
11    categoria_maior_gasto VARCHAR(100),
12    valor_maior_gasto DECIMAL(15,2)
13 ) AS $$
14 BEGIN
15     RETURN QUERY
16     WITH resumo_base AS (
17         SELECT
18             TO_CHAR(t.data_transacao, 'YYYY-MM') as mes,
19             SUM(CASE WHEN t.tipo = 'receita' THEN t.valor ELSE 0
20             END) as receitas,
21             SUM(CASE WHEN t.tipo = 'despesa' THEN t.valor ELSE 0
22             END) as despesas
23         FROM transacoes t
24         WHERE t.usuario_id = p_usuario_id
25               AND t.status = 'confirmada'
26               AND t.data_transacao BETWEEN p_data_inicio AND p_data_fim
27         GROUP BY TO_CHAR(t.data_transacao, 'YYYY-MM')
28     ),
29     maior_categoria AS (
30         SELECT DISTINCT ON (TO_CHAR(t.data_transacao, 'YYYY-MM'))
31             TO_CHAR(t.data_transacao, 'YYYY-MM') as mes,
```

```
32     FROM transacoes t
33     JOIN categorias c ON t.categoria_id = c.id
34     WHERE t.usuario_id = p_usuario_id
35           AND t.tipo = 'despesa'
36           AND t.status = 'confirmada'
37           AND t.data_transacao BETWEEN p_data_inicio AND p_data_fim
38     GROUP BY TO_CHAR(t.data_transacao, 'YYYY-MM'), c.nome
39     ORDER BY TO_CHAR(t.data_transacao, 'YYYY-MM'),
SUM(t.valor) DESC
40 )
41 SELECT
42     rb.mes,
43     rb.receitas,
44     rb.despesas,
45     (rb.receitas - rb.despesas),
46     mc.categoria,
47     mc.total_categoria
48 FROM resumo_base rb
49 LEFT JOIN maior_categoria mc ON rb.mes = mc.mes
50 ORDER BY rb.mes;
51 END;
52 $$ LANGUAGE plpgsql;
```

Listing 23: Funcao para gerar resumo financeiro

6.3 Funcoes de Validacao

6.3.1 Validacao de Limite de Credito

```
1 CREATE OR REPLACE FUNCTION validar_limite_credito()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     saldo_atual DECIMAL(15,2);
5     limite DECIMAL(15,2);
6     tipo_conta VARCHAR(30);
7 BEGIN
8     -- Buscar informacoes da conta
9     SELECT c.saldo_atual, c.limite_credito, c.tipo
10    INTO saldo_atual, limite, tipo_conta
11    FROM contas c
12    WHERE c.id = NEW.conta_id;
13
14     -- Verificar apenas para cartao de credito
15     IF tipo_conta = 'cartao_credito' AND NEW.tipo = 'despesa' THEN
16         -- Calcular novo saldo apos a transacao
17         IF (saldo_atual - NEW.valor) < -COALESCE(limite, 0) THEN
18             RAISE EXCEPTION 'Transacao excede limite de credito.
19 Limite: %, Saldo atual: %, Valor transacao: %',
20                             COALESCE(limite, 0), saldo_atual, NEW.valor;
21         END IF;
22     END IF;
```

```
22     RETURN NEW;
23
24 END;
25 $$ LANGUAGE plpgsql;
```

Listing 24: Funcao para validar limite de credito

7 Triggers

7.1 Triggers de Auditoria

7.1.1 Trigger de Log Automatico

```
1 CREATE OR REPLACE FUNCTION audit_trigger()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF TG_OP = 'DELETE' THEN
5         INSERT INTO audit.audit_logs (tabela, operacao,
6             usuario_id, dados_antigos)
7             VALUES (TG_TABLE_NAME, TG_OP, OLD.usuario_id,
8                 row_to_json(OLD));
9         RETURN OLD;
10    ELSIF TG_OP = 'UPDATE' THEN
11        INSERT INTO audit.audit_logs (tabela, operacao,
12            usuario_id, dados_antigos, dados_novos)
13            VALUES (TG_TABLE_NAME, TG_OP, NEW.usuario_id,
14                row_to_json(OLD), row_to_json(NEW));
15        RETURN NEW;
16    ELSIF TG_OP = 'INSERT' THEN
17        INSERT INTO audit.audit_logs (tabela, operacao,
18            usuario_id, dados_novos)
19            VALUES (TG_TABLE_NAME, TG_OP, NEW.usuario_id,
20                row_to_json(NEW));
21        RETURN NEW;
22    END IF;
23    RETURN NULL;
24 END;
25 $$ LANGUAGE plpgsql;
```

Listing 25: Funcao generica de auditoria

7.1.2 Aplicacao dos Triggers

```
1 -- Trigger para transacoes
2 CREATE TRIGGER trg_audit_transacoes
3     AFTER INSERT OR UPDATE OR DELETE ON transacoes
4     FOR EACH ROW EXECUTE FUNCTION audit_trigger();
5
6 -- Trigger para contas
7 CREATE TRIGGER trg_audit_contas
```

```
8      AFTER INSERT OR UPDATE OR DELETE ON contas
9      FOR EACH ROW EXECUTE FUNCTION audit_trigger();
10
11 -- Trigger para usuarios
12 CREATE TRIGGER trg_audit_usuarios
13     AFTER INSERT OR UPDATE OR DELETE ON usuarios
14     FOR EACH ROW EXECUTE FUNCTION audit_trigger();
```

Listing 26: Criacao dos triggers de auditoria

7.2 Triggers de Negocio

7.2.1 Atualizacao Automatica de Saldos

```
1 CREATE OR REPLACE FUNCTION atualizar_saldo_trigger()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
5         -- Atualizar saldo da conta origem
6         PERFORM atualizar_saldo_conta(NEW.conta_id);
7
8         -- Se for transferencia, atualizar conta destino
9         IF NEW.tipo = 'transferencia' AND NEW.conta_destino_id IS
10 NOT NULL THEN
11             PERFORM atualizar_saldo_conta(NEW.conta_destino_id);
12         END IF;
13
14     RETURN NEW;
15
16     ELSIF TG_OP = 'DELETE' THEN
17         -- Atualizar saldo da conta origem
18         PERFORM atualizar_saldo_conta(OLD.conta_id);
19
20         -- Se for transferencia, atualizar conta destino
21         IF OLD.tipo = 'transferencia' AND OLD.conta_destino_id IS
22 NOT NULL THEN
23             PERFORM atualizar_saldo_conta(OLD.conta_destino_id);
24         END IF;
25
26     RETURN OLD;
27
28     END IF;
29     RETURN NULL;
30 END;
31 $$ LANGUAGE plpgsql;
32
33 -- Aplicar trigger
34 CREATE TRIGGER trg_atualizar_saldo
35     AFTER INSERT OR UPDATE OR DELETE ON transacoes
36     FOR EACH ROW EXECUTE FUNCTION atualizar_saldo_trigger();
```

Listing 27: Trigger para atualizar saldo automaticamente

7.2.2 Validacao de Integridade

```
1 CREATE OR REPLACE FUNCTION validar_integridade_trigger()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Validar que usuario pertence ao grupo da transacao
5     IF NEW.grupo_id IS NOT NULL THEN
6         IF NOT EXISTS (
7             SELECT 1 FROM usuario_grupos
8             WHERE usuario_id = NEW.usuario_id
9                 AND grupo_id = NEW.grupo_id
10                AND ativo = true
11         ) THEN
12             RAISE EXCEPTION 'Usuario nao pertence ao grupo
13 especificado';
14         END IF;
15     END IF;
16
17     -- Validar que conta pertence ao usuario ou grupo
18     IF NOT EXISTS (
19         SELECT 1 FROM contas c
20         WHERE c.id = NEW.conta_id
21             AND (c.usuario_id = NEW.usuario_id OR
22                 (c.compartilhada = true AND c.grupo_id =
23 NEW.grupo_id))
24     ) THEN
25         RAISE EXCEPTION 'Usuario nao tem acesso a conta
26 especificada';
27     END IF;
28
29     RETURN NEW;
30 END;
31 $$ LANGUAGE plpgsql;
32
33 -- Aplicar trigger
34 CREATE TRIGGER trg_validar_integridade
35 BEFORE INSERT OR UPDATE ON transacoes
36 FOR EACH ROW EXECUTE FUNCTION validar_integridade_trigger();
```

Listing 28: Trigger para validacoes de integridade

7.3 Triggers de Performance

7.3.1 Atualizacao de Views Materializadas

```
1 CREATE OR REPLACE FUNCTION refresh_materialized_views()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Refresh da view de resumo financeiro (em background)
5     PERFORM pg_notify('refresh_mv', 'resumo_financeiro_mensal');
6
```

```
7      RETURN COALESCE(NEW, OLD);
8  END;
9  $$ LANGUAGE plpgsql;
10
11  -- Aplicar trigger
12  CREATE TRIGGER trg_refresh_mv_transacoes
13      AFTER INSERT OR UPDATE OR DELETE ON transacoes
14      FOR EACH STATEMENT EXECUTE FUNCTION
      refresh_materialized_views();
```

Listing 29: Trigger para refresh de views materializadas

8 Indices e Otimizacao

8.1 Estrategia de Indexacao

8.1.1 Indices Primarios

```
1  -- Indices para consultas frequentes por usuario
2  CREATE INDEX idx_transacoes_usuario_data_status
3      ON transacoes(usuario_id, data_transacao DESC, status);
4
5  CREATE INDEX idx_contas_usuario_ativa_tipo
6      ON contas(usuario_id, ativa, tipo);
7
8  -- Indices para consultas de grupo
9  CREATE INDEX idx_transacoes_grupo_data
10     ON transacoes(grupo_id, data_transacao DESC)
11     WHERE grupo_id IS NOT NULL;
12
13  -- Indices para relatorios
14  CREATE INDEX idx_transacoes_categoria_data
15     ON transacoes(categoria_id, data_transacao DESC, status);
16
17  -- Indices para busca textual
18  CREATE INDEX idx_transacoes_descricao_gin
19     ON transacoes USING gin(to_tsvector('portuguese', descricao));
```

Listing 30: Indices essenciais para performance

8.1.2 Indices Condicionais

```
1  -- Apenas transacoes ativas
2  CREATE INDEX idx_transacoes_ativas
3      ON transacoes(usuario_id, data_transacao DESC)
4      WHERE status = 'confirmada';
5
6  -- Apenas contas ativas
7  CREATE INDEX idx_contas_ativas_saldo
8      ON contas(usuario_id, saldo_atual)
```

```
9      WHERE ativa = true;
10
11 -- Apenas usuarios verificados
12 CREATE INDEX idx_usuarios_verificados
13     ON usuarios(email)
14     WHERE email_verificado = true AND ativo = true;
15
16 -- Recorrencias ativas
17 CREATE INDEX idx_recorrencias_proxima
18     ON recorrencias(proxima_execucao)
19     WHERE ativa = true;
```

Listing 31: Indices otimizados com condicoes

8.2 Monitoramento de Performance

8.2.1 Views de Monitoramento

```
1 -- View para consultas lentas
2 CREATE VIEW slow_queries AS
3 SELECT
4     query,
5     calls,
6     total_time,
7     mean_time,
8     rows,
9     100.0 * shared_blks_hit / nullif(shared_blks_hit +
10     shared_blks_read, 0) AS hit_percent
11 FROM pg_stat_statements
12 WHERE calls > 100
13 ORDER BY mean_time DESC;
14
15 -- View para uso de indices
16 CREATE VIEW index_usage AS
17 SELECT
18     schemaname,
19     tablename,
20     indexname,
21     idx_tup_read,
22     idx_tup_fetch,
23     idx_scan,
24     pg_size_pretty(pg_relation_size(indexrelid)) as size
25 FROM pg_stat_user_indexes
26 ORDER BY idx_scan DESC;
27
28 -- View para tabelas grandes
29 CREATE VIEW table_sizes AS
30 SELECT
31     schemaname,
```

```
32      pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename))
      as size,
33      pg_total_relation_size(schemaname||'.'||tablename) as bytes
34 FROM pg_tables
35 WHERE schemaname = 'public'
36 ORDER BY bytes DESC;
```

Listing 32: Views para analise de performance

8.3 Otimizacoes Avancadas

8.3.1 Particao de Tabelas

```
1  -- Particionar transacoes por ano para melhor performance
2  CREATE TABLE transacoes_2024 PARTITION OF transacoes
3      FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
4
5  CREATE TABLE transacoes_2025 PARTITION OF transacoes
6      FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
7
8  -- Funcao para criar particoes automaticamente
9  CREATE OR REPLACE FUNCTION criar_particao_mensal()
10 RETURNS void AS $$
11 DECLARE
12     ano INTEGER := EXTRACT(YEAR FROM CURRENT_DATE);
13     mes INTEGER := EXTRACT(MONTH FROM CURRENT_DATE);
14     data_inicio DATE;
15     data_fim DATE;
16     nome_tabela TEXT;
17 BEGIN
18     data_inicio := DATE_TRUNC('month', CURRENT_DATE);
19     data_fim := data_inicio + INTERVAL '1 month';
20     nome_tabela := 'transacoes_' || ano || '_' || LPAD(mes::TEXT,
21     2, '0');
22
23     EXECUTE format('CREATE TABLE IF NOT EXISTS %I PARTITION OF
24     transacoes
25                     FOR VALUES FROM (%L) TO (%L)',
26                     nome_tabela, data_inicio, data_fim);
27 END;
28 $$ LANGUAGE plpgsql;
```

Listing 33: Estrategia de particao para transacoes

8.3.2 Cache e Buffers

```
1  -- Configuracoes no postgresql.conf
2  -- shared_buffers = 256MB (25% da RAM)
3  -- effective_cache_size = 1GB (75% da RAM)
```

```
4 -- work_mem = 4MB
5 -- maintenance_work_mem = 64MB
6 -- checkpoint_completion_target = 0.9
7 -- wal_buffers = 16MB
8 -- default_statistics_target = 100
9
10 -- Query para verificar configuracoes atuais
11 SELECT name, setting, unit, context
12 FROM pg_settings
13 WHERE name IN (
14     'shared_buffers',
15     'effective_cache_size',
16     'work_mem',
17     'maintenance_work_mem'
18 );
```

Listing 34: Configuracoes recomendadas para PostgreSQL

8.4 Analise e Manutencao

8.4.1 Scripts de Analise

```
1 -- Atualizar estatisticas
2 ANALYZE;
3
4 -- Verificar bloat em tabelas
5 SELECT
6     tablename,
7     pg_size_pretty(pg_total_relation_size(tablename::regclass)) as
8     size,
9     round(100 * (pg_total_relation_size(tablename::regclass) -
10     pg_relation_size(tablename::regclass))::numeric /
11     pg_total_relation_size(tablename::regclass), 2)
12     as bloat_pct
13 FROM pg_tables
14 WHERE schemaname = 'public'
15 AND pg_total_relation_size(tablename::regclass) > 1000000
16 ORDER BY pg_total_relation_size(tablename::regclass) DESC;
17
18 -- Verificar indices nao utilizados
19 SELECT
20     schemaname,
21     tablename,
22     indexname,
23     idx_scan,
24     pg_size_pretty(pg_relation_size(indexname::regclass)) as size
25 FROM pg_stat_user_indexes
26 WHERE idx_scan = 0
27 AND schemaname = 'public'
28 ORDER BY pg_relation_size(indexname::regclass) DESC;
```

Listing 35: Scripts para analise de performance

8.4.2 Manutencao Automatica

```
1 CREATE OR REPLACE FUNCTION manutencao_automatica()
2 RETURNS void AS $$
3 BEGIN
4     -- Vacuum analize em tabelas principais
5     VACUUM ANALYZE usuarios;
6     VACUUM ANALYZE grupos;
7     VACUUM ANALYZE contas;
8     VACUUM ANALYZE categorias;
9     VACUUM ANALYZE transacoes;
10
11     -- Refresh views materializadas
12     REFRESH MATERIALIZED VIEW CONCURRENTLY
    resumo_financeiro_mensal;
13
14     -- Limpar logs antigos (>90 dias)
15     DELETE FROM audit.audit_logs
16     WHERE timestamp_operacao < CURRENT_DATE - INTERVAL '90 days';
17
18     -- Limpar sessoes expiradas
19     DELETE FROM sessoes
20     WHERE data_expiracao < CURRENT_TIMESTAMP;
21
22     -- Log da manutencao
23     INSERT INTO audit.audit_logs (tabela, operacao, dados_novos)
24     VALUES ('sistema', 'MAINTENANCE',
25             jsonb_build_object('timestamp', CURRENT_TIMESTAMP,
26                                'status', 'concluida'));
27 END;
28 $$ LANGUAGE plpgsql;
```

Listing 36: Funcao para manutencao automatica

9 Seguranca

9.1 Controle de Acesso

9.1.1 Roles e Permissoes

```
1 -- Role para aplicacao (conexao principal)
2 CREATE ROLE finops_app WITH LOGIN PASSWORD 'senha_forte_aqui';
3
4 -- Role para backups
5 CREATE ROLE finops_backup WITH LOGIN PASSWORD 'senha_backup_aqui';
6
```

```
7  -- Role para leitura (relatorios)
8  CREATE ROLE finops_readonly WITH LOGIN PASSWORD
   'senha_readonly_aqui';
9
10 -- Role para administracao
11 CREATE ROLE finops_admin WITH LOGIN SUPERUSER PASSWORD
   'senha_admin_aqui';
12
13 -- Conceder permissoes basicas para aplicacao
14 GRANT CONNECT ON DATABASE finops TO finops_app;
15 GRANT USAGE ON SCHEMA public TO finops_app;
16 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
   public TO finops_app;
17 GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO
   finops_app;
18 GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO finops_app;
19
20 -- Permissoes para auditoria
21 GRANT USAGE ON SCHEMA audit TO finops_app;
22 GRANT INSERT ON audit.audit_logs TO finops_app;
23
24 -- Permissoes para readonly
25 GRANT CONNECT ON DATABASE finops TO finops_readonly;
26 GRANT USAGE ON SCHEMA public TO finops_readonly;
27 GRANT SELECT ON ALL TABLES IN SCHEMA public TO finops_readonly;
28
29 -- Permissoes para backup
30 GRANT CONNECT ON DATABASE finops TO finops_backup;
31 GRANT USAGE ON SCHEMA public, audit TO finops_backup;
32 GRANT SELECT ON ALL TABLES IN SCHEMA public, audit TO
   finops_backup;
```

Listing 37: Criacao de roles de segurança

9.1.2 Row Level Security (RLS)

```
1  -- Habilitar RLS nas tabelas principais
2  ALTER TABLE usuarios ENABLE ROW LEVEL SECURITY;
3  ALTER TABLE contas ENABLE ROW LEVEL SECURITY;
4  ALTER TABLE transacoes ENABLE ROW LEVEL SECURITY;
5  ALTER TABLE categorias ENABLE ROW LEVEL SECURITY;
6
7  -- Politica para usuarios (apenas dados proprios)
8  CREATE POLICY usuarios_self_access ON usuarios
9     FOR ALL TO finops_app
10    USING (id = current_setting('app.current_user_id')::uuid);
11
12 -- Politica para contas (proprias ou de grupo)
13 CREATE POLICY contas_access ON contas
14    FOR ALL TO finops_app
15    USING (
```

```
16      usuario_id = current_setting('app.current_user_id')::uuid
17  OR
18      (compartilhada = true AND grupo_id IN (
19          SELECT grupo_id FROM usuario_grupos
20          WHERE usuario_id =
21              current_setting('app.current_user_id')::uuid
22              AND ativo = true
23      ))
24  );
25
26  -- Politica para transacoes
27  CREATE POLICY transacoes_access ON transacoes
28  FOR ALL TO finops_app
29  USING (
30      usuario_id = current_setting('app.current_user_id')::uuid
31  OR
32      grupo_id IN (
33          SELECT grupo_id FROM usuario_grupos
34          WHERE usuario_id =
35              current_setting('app.current_user_id')::uuid
36              AND ativo = true
37      )
38  );
39
40  -- Politica para categorias
41  CREATE POLICY categorias_access ON categorias
42  FOR ALL TO finops_app
43  USING (
44      sistema = true OR
45      usuario_id = current_setting('app.current_user_id')::uuid
46  OR
47      grupo_id IN (
48          SELECT grupo_id FROM usuario_grupos
49          WHERE usuario_id =
50              current_setting('app.current_user_id')::uuid
51              AND ativo = true
52      )
53  );
```

Listing 38: Politicas de seguranca por linha

9.2 Criptografia

9.2.1 Criptografia de Dados Sensiveis

```
1  -- Extensao para criptografia
2  CREATE EXTENSION IF NOT EXISTS pgcrypto;
3
4  -- Funcao para hash de senhas
5  CREATE OR REPLACE FUNCTION hash_senha(senha TEXT)
6  RETURNS TEXT AS $$
```



```
7 BEGIN
8     RETURN crypt(senha, gen_salt('bf', 12));
9 END;
10 $$ LANGUAGE plpgsql;
11
12 -- Funcao para verificar senha
13 CREATE OR REPLACE FUNCTION verificar_senha(senha TEXT, hash TEXT)
14 RETURNS BOOLEAN AS $$
15 BEGIN
16     RETURN hash = crypt(senha, hash);
17 END;
18 $$ LANGUAGE plpgsql;
19
20 -- Funcao para criptografar dados sensiveis
21 CREATE OR REPLACE FUNCTION criptografar_dado(dado TEXT, chave TEXT)
22 RETURNS TEXT AS $$
23 BEGIN
24     RETURN encode(pgp_sym_encrypt(dado, chave), 'base64');
25 END;
26 $$ LANGUAGE plpgsql;
27
28 -- Funcao para descriptografar dados
29 CREATE OR REPLACE FUNCTION descriptografar_dado(dado_criptografado
30 TEXT, chave TEXT)
31 RETURNS TEXT AS $$
32 BEGIN
33     RETURN pgp_sym_decrypt(decode(dado_criptografado, 'base64'),
34     chave);
35 EXCEPTION WHEN OTHERS THEN
36     RETURN NULL;
37 END;
38 $$ LANGUAGE plpgsql;
```

Listing 39: Funcoes para criptografia

9.3 Auditoria e Logging

9.3.1 Log de Acessos

```
1 CREATE TABLE audit.login_logs (
2     id BIGSERIAL PRIMARY KEY,
3     usuario_id UUID REFERENCES usuarios(id),
4     ip_origem INET NOT NULL,
5     user_agent TEXT,
6     sucesso BOOLEAN NOT NULL,
7     motivo_falha TEXT,
8     timestamp_login TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
10
11 -- Funcao para log de login
12 CREATE OR REPLACE FUNCTION registrar_login(
```

```
13     p_usuario_id UUID,  
14     p_ip INET,  
15     p_user_agent TEXT,  
16     p_sucesso BOOLEAN,  
17     p_motivo_falha TEXT DEFAULT NULL  
18 )  
19 RETURNS void AS $$  
20 BEGIN  
21     INSERT INTO audit.login_logs (usuario_id, ip_origem,  
22     user_agent, sucesso, motivo_falha)  
23     VALUES (p_usuario_id, p_ip, p_user_agent, p_sucesso,  
24     p_motivo_falha);  
25 END;  
26 $$ LANGUAGE plpgsql;
```

Listing 40: Trigger para log de acessos

9.3.2 Deteccao de Anomalias

```
1 CREATE OR REPLACE FUNCTION  
2     detectar_atividade_suspeita(p_usuario_id UUID)  
3 RETURNS TABLE (  
4     tipo_alerta TEXT,  
5     descricao TEXT,  
6     severidade INTEGER  
7 ) AS $$  
8 BEGIN  
9     -- Multiplos logins de IPs diferentes  
10    IF EXISTS (  
11        SELECT 1 FROM audit.login_logs  
12        WHERE usuario_id = p_usuario_id  
13              AND timestamp_login > CURRENT_TIMESTAMP - INTERVAL '1  
14              hour'  
15              AND sucesso = true  
16              GROUP BY ip_origem  
17              HAVING COUNT(*) > 5  
18    ) THEN  
19        RETURN QUERY SELECT 'MULTIPLOS_IPS'::TEXT, 'Multiplos  
20        logins de IPs diferentes'::TEXT, 3;  
21    END IF;  
22  
23    -- Transacoes em valores muito altos  
24    IF EXISTS (  
25        SELECT 1 FROM transacoes t  
26        WHERE t.usuario_id = p_usuario_id  
27              AND t.data_transacao > CURRENT_DATE - INTERVAL '1 day'  
28              AND t.valor > (  
29          SELECT COALESCE(AVG(valor) * 10, 1000)  
30          FROM transacoes t2  
31          WHERE t2.usuario_id = p_usuario_id
```

```

29         AND t2.data_transacao > CURRENT_DATE - INTERVAL
30         '30 days'
31     ) THEN
32         RETURN QUERY SELECT 'VALOR_ALTO'::TEXT, 'Transacao com
33         valor muito acima da media'::TEXT, 2;
34     END IF;
35
36     -- Muitas transacoes em pouco tempo
37     IF (
38         SELECT COUNT(*) FROM transacoes
39         WHERE usuario_id = p_usuario_id
40         AND created_at > CURRENT_TIMESTAMP - INTERVAL '10
41         minutes'
42     ) > 20 THEN
43         RETURN QUERY SELECT 'MUITAS_TRANSACOES'::TEXT, 'Muitas
44         transacoes em pouco tempo'::TEXT, 2;
45     END IF;
46 END;
47 $$ LANGUAGE plpgsql;

```

Listing 41: Funcao para detectar atividades suspeitas

9.4 Configuracoes de Seguranca

9.4.1 Parametros do PostgreSQL

```

1  -- No postgresql.conf
2  -- ssl = on
3  -- ssl_cert_file = 'server.crt'
4  -- ssl_key_file = 'server.key'
5  -- ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL'
6  -- password_encryption = scram-sha-256
7  -- log_connections = on
8  -- log_disconnections = on
9  -- log_statement = 'mod'
10 -- log_line_prefix = '%t [%p]: [%l-1]
11     user=%u,db=%d,app=%a,client=%h '
12
13 -- No pg_hba.conf
14 -- local    finops    finops_app          scram-sha-256
15 -- host     finops    finops_app  127.0.0.1/32  scram-sha-256
16 -- hostssl  finops    finops_app  0.0.0.0/0      scram-sha-256

```

Listing 42: Configuracoes recomendadas de seguranca

9.5 Backup Seguro

```

1 #!/bin/bash
2 # backup_seguro.sh

```

```
3
4 # Configuracoes
5 DB_NAME="finops"
6 BACKUP_DIR="/var/backups/finops"
7 DATE=$(date +%Y%m%d_%H%M%S)
8 BACKUP_FILE="$BACKUP_DIR/finops_backup_$DATE.sql.gz.gpg"
9
10 # Criar diretorio se nao existir
11 mkdir -p $BACKUP_DIR
12
13 # Realizar backup com compressao e criptografia
14 pg_dump $DB_NAME | gzip | gpg --cipher-algo AES256 --compress-algo
15 1 \
16     --symmetric --output $BACKUP_FILE
17
18 # Verificar se backup foi criado
19 if [ -f "$BACKUP_FILE" ]; then
20     echo "Backup criado com sucesso: $BACKUP_FILE"
21
22     # Remover backups antigos (manter apenas 30 dias)
23     find $BACKUP_DIR -name "finops_backup_*.sql.gz.gpg" -mtime +30
24     -delete
25 else
26     echo "ERRO: Falha ao criar backup"
27     exit 1
28 fi
```

Listing 43: Script de backup com criptografia

10 Backup e Recovery

10.1 Estrategia de Backup

10.1.1 Tipos de Backup

Backup Completo Diario

```
1 #!/bin/bash
2 # backup_completo.sh
3
4 DB_NAME="finops"
5 BACKUP_DIR="/var/backups/finops/completo"
6 DATE=$(date +%Y%m%d_%H%M%S)
7 LOGFILE="/var/log/finops_backup.log"
8
9 echo "$(date): Iniciando backup completo" >> $LOGFILE
10
11 # Backup com compressao
12 pg_dump -h localhost -U finops_backup -d $DB_NAME \
13     --format=custom --compress=9 \
14     --file="$BACKUP_DIR/finops_full_$DATE.backup"
15
```

```
16 if [ $? -eq 0 ]; then
17     echo "$(date): Backup completo concluido com sucesso" >>
    $LOGFILE
18
19     # Testar integridade do backup
20     pg_restore --list "$BACKUP_DIR/finops_full_$(date +%Y%m%d_%H%M%S).backup" >
    /dev/null
21
22     if [ $? -eq 0 ]; then
23         echo "$(date): Backup validado com sucesso" >> $LOGFILE
24     else
25         echo "$(date): ERRO: Backup corrompido" >> $LOGFILE
26         exit 1
27     fi
28 else
29     echo "$(date): ERRO: Falha no backup completo" >> $LOGFILE
30     exit 1
31 fi
32
33 # Limpeza de backups antigos (manter 7 dias)
34 find $BACKUP_DIR -name "finops_full_*.backup" -mtime +7 -delete
```

Listing 44: Script de backup completo

Backup Incremental

```
1 -- No postgresql.conf
2 -- wal_level = replica
3 -- archive_mode = on
4 -- archive_command = 'cp %p /var/backups/finops/wal/%f'
5 -- max_wal_senders = 3
6 -- checkpoint_timeout = 5min
7 -- max_wal_size = 1GB
8 -- min_wal_size = 80MB
9
10 -- Script para backup incremental
11 #!/bin/bash
12 # backup_incremental.sh
13
14 PGDATA="/var/lib/postgresql/data"
15 BACKUP_DIR="/var/backups/finops/incremental"
16 DATE=$(date +%Y%m%d_%H%M%S)
17
18 # Iniciar backup base
19 pg_basebackup -h localhost -U finops_backup \
20     -D "$BACKUP_DIR/base_$(date +%Y%m%d_%H%M%S)" \
21     --format=tar --gzip --progress \
22     --checkpoint=fast
```

Listing 45: Configuracao de WAL archiving

10.2 Procedures de Recovery

10.2.1 Recovery Completo

```
1 #!/bin/bash
2 # restore_completo.sh
3
4 if [ -z "$1" ]; then
5     echo "Uso: $0 <arquivo_backup>"
6     exit 1
7 fi
8
9 BACKUP_FILE="$1"
10 DB_NAME="finops"
11 TEMP_DB="finops_restore_$(date +%s)"
12
13 echo "Iniciando restore do backup: $BACKUP_FILE"
14
15 # Criar banco temporario
16 createdb -U postgres "$TEMP_DB"
17
18 # Restaurar dados
19 pg_restore -h localhost -U postgres -d "$TEMP_DB" \
20     --verbose --clean --if-exists "$BACKUP_FILE"
21
22 if [ $? -eq 0 ]; then
23     echo "Restore concluido no banco temporario: $TEMP_DB"
24     echo "Para aplicar:"
25     echo "1. Pare a aplicacao"
26     echo "2. Renomeie o banco atual: ALTER DATABASE $DB_NAME
27     RENAME TO ${DB_NAME}_old;"
28     echo "3. Renomeie o banco restaurado: ALTER DATABASE $TEMP_DB
29     RENAME TO $DB_NAME;"
30     echo "4. Reinicie a aplicacao"
31 else
32     echo "ERRO no restore"
33     dropdb -U postgres "$TEMP_DB"
34     exit 1
35 fi
```

Listing 46: Script de restore completo

10.2.2 Point-in-Time Recovery

```
1 #!/bin/bash
2 # pitr_recovery.sh
3
4 if [ -z "$1" ] || [ -z "$2" ]; then
5     echo "Uso: $0 <backup_base> <timestamp>"
6     echo "Exemplo: $0 /path/backup.tar '2024-01-15 14:30:00'"
7     exit 1
```

```
8 fi
9
10 BACKUP_BASE="$1"
11 TARGET_TIME="$2"
12 RECOVERY_DIR="/var/lib/postgresql/recovery"
13 WAL_DIR="/var/backups/finops/wal"
14
15 # Parar PostgreSQL
16 systemctl stop postgresql
17
18 # Limpar diretorio de dados
19 rm -rf /var/lib/postgresql/data/*
20
21 # Extrair backup base
22 tar -xzf "$BACKUP_BASE" -C /var/lib/postgresql/data/
23
24 # Criar arquivo de recovery
25 cat > /var/lib/postgresql/data/recovery.conf << EOF
26 restore_command = 'cp $WAL_DIR/%f %p'
27 recovery_target_time = '$TARGET_TIME'
28 recovery_target_timeline = 'latest'
29 EOF
30
31 # Iniciar PostgreSQL em modo recovery
32 systemctl start postgresql
33
34 echo "Recovery iniciado para o momento: $TARGET_TIME"
35 echo "Monitore os logs: tail -f /var/log/postgresql/postgresql.log"
```

Listing 47: Recovery para um momento especifico

10.3 Monitoramento de Backup

10.3.1 Verificacao Automatica

```
1 #!/bin/bash
2 # verificar_backups.sh
3
4 BACKUP_DIR="/var/backups/finops"
5 LOGFILE="/var/log/finops_backup_check.log"
6 EMAIL_ADMIN="admin@finops.com"
7
8 echo "$(date): Iniciando verificacao de backups" >> $LOGFILE
9
10 # Verificar backup mais recente
11 ULTIMO_BACKUP=$(find $BACKUP_DIR/completo -name
12     "finops_full_*.backup" -mtime -1 | head -1)
13
14 if [ -z "$ULTIMO_BACKUP" ]; then
15     echo "$(date): ERRO: Nenhum backup encontrado nas ultimas 24h"
16     >> $LOGFILE
```

```
15     echo "ALERTA: Backup nao encontrado" | mail -s "FinOps Backup
16     Alert" $EMAIL_ADMIN
17     exit 1
18 fi
19 # Testar integridade
20 pg_restore --list "$ULTIMO_BACKUP" > /dev/null 2>&1
21
22 if [ $? -eq 0 ]; then
23     echo "$(date): Backup integro: $ULTIMO_BACKUP" >> $LOGFILE
24 else
25     echo "$(date): ERRO: Backup corrompido: $ULTIMO_BACKUP" >>
26     $LOGFILE
27     echo "ALERTA: Backup corrompido" | mail -s "FinOps Backup
28     Alert" $EMAIL_ADMIN
29     exit 1
30 fi
31
32 # Verificar espaco em disco
33 ESPACO_LIVRE=$(df $BACKUP_DIR | awk 'NR==2 {print $4}')
34 LIMITE_MINIMO=1048576 # 1GB em KB
35
36 if [ $ESPACO_LIVRE -lt $LIMITE_MINIMO ]; then
37     echo "$(date): AVISO: Pouco espaco em disco:
38     ${ESPACO_LIVRE}KB" >> $LOGFILE
39     echo "AVISO: Pouco espaco para backups" | mail -s "FinOps
40     Storage Alert" $EMAIL_ADMIN
41 fi
42
43 echo "$(date): Verificacao concluida com sucesso" >> $LOGFILE
```

Listing 48: Script de verificacao de backups

10.4 Disaster Recovery

10.4.1 Plano de Contingencia

```
1  -- PLANO DE DISASTER RECOVERY FINOPS
2  -- =====
3
4  -- FASE 1: AVALIACAO DO DANO
5  -- 1. Verificar se servidor PostgreSQL responde
6  -- 2. Testar conectividade de rede
7  -- 3. Verificar integridade dos discos
8  -- 4. Avaliar logs de erro
9
10 -- FASE 2: RECOVERY DE EMERGENCIA
11 -- 1. Provisionar novo servidor se necessario
12 -- 2. Instalar PostgreSQL na mesma versao
13 -- 3. Configurar parametros basicos
14 -- 4. Restaurar backup mais recente
```



```
15
16 -- Script de recovery de emergencia
17 #!/bin/bash
18 # emergency_recovery.sh
19
20 echo "=== FinOps Emergency Recovery ==="
21 echo "Este script ira restaurar o banco de dados"
22 echo "Confirme antes de prosseguir!"
23 read -p "Continuar? (y/N): " confirm
24
25 if [ "$confirm" != "y" ]; then
26     echo "Recovery cancelado"
27     exit 0
28 fi
29
30 # Encontrar backup mais recente
31 ULTIMO_BACKUP=$(find /var/backups/finops/completo -name
    "finops_full_*.backup" | sort | tail -1)
32
33 if [ -z "$ULTIMO_BACKUP" ]; then
34     echo "ERRO: Nenhum backup encontrado"
35     exit 1
36 fi
37
38 echo "Usando backup: $ULTIMO_BACKUP"
39
40 # Recriar banco
41 dropdb --if-exists finops
42 createdb finops
43
44 # Restaurar dados
45 pg_restore -d finops "$ULTIMO_BACKUP"
46
47 echo "Recovery concluido"
48 echo "Verificar logs e testar aplicacao antes de colocar em
    producao"
```

Listing 49: Procedimento de disaster recovery

10.4.2 Replicacao para Alta Disponibilidade

```
1 -- No servidor master (postgresql.conf)
2 -- wal_level = replica
3 -- max_wal_senders = 3
4 -- wal_keep_segments = 64
5 -- synchronous_standby_names = 'standby1'
6
7 -- No pg_hba.conf do master
8 -- host replication finops_repl standby_ip/32 md5
9
10 -- Script para configurar standby
```

```
11 #!/bin/bash
12 # setup_standby.sh
13
14 MASTER_HOST="master_ip"
15 STANDBY_DATA="/var/lib/postgresql/standby"
16
17 # Parar PostgreSQL no standby
18 systemctl stop postgresql
19
20 # Backup inicial do master
21 pg_basebackup -h $MASTER_HOST -U finops_repl \
22     -D $STANDBY_DATA --write-recovery-conf \
23     --checkpoint=fast
24
25 # Configurar recovery.conf
26 cat >> $STANDBY_DATA/recovery.conf << EOF
27 standby_mode = 'on'
28 primary_conninfo = 'host=$MASTER_HOST port=5432 user=finops_repl'
29 trigger_file = '/var/lib/postgresql/failover'
30 EOF
31
32 # Iniciar standby
33 systemctl start postgresql
34
35 echo "Standby configurado. Para failover, crie o arquivo:"
36 echo "touch /var/lib/postgresql/failover"
```

Listing 50: Configuracao de replica

11 Performance e Otimizacao

11.1 Monitoramento de Performance

11.1.1 Metricas Essenciais

```
1 -- CPU e memoria por query
2 SELECT
3     query,
4     calls,
5     total_time,
6     mean_time,
7     stddev_time,
8     rows,
9     100.0 * shared_blks_hit / nullif(shared_blks_hit +
10     shared_blks_read, 0) AS hit_percent
11 FROM pg_stat_statements
12 ORDER BY mean_time DESC
13 LIMIT 10;
14 -- Bloqueios ativos
```

```
15 SELECT
16     pg_stat_activity.pid,
17     pg_stat_activity.username,
18     pg_stat_activity.query,
19     pg_stat_activity.state,
20     pg_locks.mode,
21     pg_locks.locktype,
22     pg_locks.relation::regclass
23 FROM pg_stat_activity
24 JOIN pg_locks ON pg_stat_activity.pid = pg_locks.pid
25 WHERE pg_stat_activity.state = 'active';
26
27 -- Conexoes ativas por usuario
28 SELECT
29     username,
30     count(*) as active_connections,
31     max(state) as max_state
32 FROM pg_stat_activity
33 WHERE state != 'idle'
34 GROUP BY username
35 ORDER BY active_connections DESC;
```

Listing 51: Queries para monitoramento

11.1.2 Dashboard de Monitoramento

```
1 CREATE VIEW dashboard_performance AS
2 WITH estatisticas_gerais AS (
3     SELECT
4         (SELECT count(*) FROM usuarios WHERE ativo = true) as
         usuarios_ativos,
5         (SELECT count(*) FROM transacoes WHERE data_transacao =
         CURRENT_DATE) as transacoes_hoje,
6         (SELECT count(*) FROM contas WHERE ativa = true) as
         contas_ativas,
7         (SELECT pg_size_pretty(pg_database_size('finops')))) as
         tamanho_bd
8 ),
9 performance_queries AS (
10     SELECT
11         count(*) as queries_executadas,
12         avg(mean_time) as tempo_medio,
13         max(mean_time) as tempo_maximo
14     FROM pg_stat_statements
15     WHERE calls > 0
16 ),
17 uso_recursos AS (
18     SELECT
19         round(100.0 * sum(shared_blks_hit) /
         nullif(sum(shared_blks_hit + shared_blks_read), 0), 2) as
         cache_hit_ratio,
```

```
20         count(*) as conexoes_ativas
21     FROM pg_stat_activity
22     WHERE state = 'active'
23 )
24 SELECT
25     eg.*,
26     pq.queries_executadas,
27     pq.tempo_medio,
28     pq.tempo_maximo,
29     ur.cache_hit_ratio,
30     ur.conexoes_ativas,
31     CURRENT_TIMESTAMP as ultima_atualizacao
32 FROM estatisticas_gerais eg, performance_queries pq, uso_recursos
    ur;
```

Listing 52: View consolidada para dashboard

11.2 Otimizacoes de Query

11.2.1 Analise de Queries Lentas

```
1  -- Habilitar log de queries lentas
2  -- No postgresql.conf:
3  -- log_min_duration_statement = 1000 # log queries > 1s
4  -- log_statement = 'all'
5  -- log_duration = on
6
7  -- Query para analisar queries mais lentas
8  SELECT
9      substring(query, 1, 50) as query_inicio,
10     calls,
11     total_time,
12     mean_time,
13     rows,
14     100.0 * shared_blks_hit / nullif(shared_blks_hit +
15     shared_blks_read, 0) AS hit_percent
16 FROM pg_stat_statements
17 WHERE mean_time > 100 -- queries com media > 100ms
18 ORDER BY mean_time DESC;
19
20 -- Analisar plano de execucao de query especifica
21 EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
22 SELECT t.*, c.nome as categoria_nome
23 FROM transacoes t
24 JOIN categorias c ON t.categoria_id = c.id
25 WHERE t.usuario_id = 'uuid_exemplo'
26     AND t.data_transacao >= '2024-01-01'
27 ORDER BY t.data_transacao DESC;
```

Listing 53: Identificacao de queries problematicas

11.2.2 Otimizações Específicas

```
1  -- Query otimizada para extrato mensal
2  CREATE INDEX IF NOT EXISTS idx_transacoes_usuario_data_categoria
3      ON transacoes(usuario_id, data_transacao DESC, categoria_id)
4      WHERE status = 'confirmada';
5
6  -- View materializada para resumos mensais rápidos
7  CREATE MATERIALIZED VIEW mv_resumo_mensal AS
8  SELECT
9      usuario_id,
10     DATE_TRUNC('month', data_transacao) as mes,
11     categoria_id,
12     SUM(CASE WHEN tipo = 'receita' THEN valor ELSE 0 END) as
13     receitas,
14     SUM(CASE WHEN tipo = 'despesa' THEN valor ELSE 0 END) as
15     despesas,
16     COUNT(*) as total_transacoes
17 FROM transacoes
18 WHERE status = 'confirmada'
19 GROUP BY usuario_id, DATE_TRUNC('month', data_transacao),
20          categoria_id;
21
22 -- Índice na view materializada
23 CREATE UNIQUE INDEX idx_mv_resumo_mensal
24     ON mv_resumo_mensal(usuario_id, mes, categoria_id);
25
26 -- Query otimizada usando a view
27 SELECT
28     c.nome as categoria,
29     sum(rm.receitas) as total_receitas,
30     sum(rm.despesas) as total_despesas
31 FROM mv_resumo_mensal rm
32 JOIN categorias c ON rm.categoria_id = c.id
33 WHERE rm.usuario_id = $1
34     AND rm.mes >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '6
35     months')
36 GROUP BY c.nome
37 ORDER BY sum(rm.despesas) DESC;
```

Listing 54: Queries otimizadas para casos comuns

11.3 Manutenção Preventiva

11.3.1 Scripts de Manutenção

```
1 #!/bin/bash
2 # manutencao_semanal.sh
3
4 LOGFILE="/var/log/finops_manutencao.log"
5
```

```
6 echo "$(date): Iniciando manutencao semanal" >> $LOGFILE
7
8 # 1. VACUUM ANALYZE em tabelas principais
9 echo "$(date): Executando VACUUM ANALYZE" >> $LOGFILE
10 psql -d finops -c "VACUUM ANALYZE usuarios;"
11 psql -d finops -c "VACUUM ANALYZE grupos;"
12 psql -d finops -c "VACUUM ANALYZE contas;"
13 psql -d finops -c "VACUUM ANALYZE categorias;"
14 psql -d finops -c "VACUUM ANALYZE transacoes;"
15
16 # 2. Reindex tabelas com muita atividade
17 echo "$(date): Reindexando tabelas" >> $LOGFILE
18 psql -d finops -c "REINDEX TABLE transacoes;"
19 psql -d finops -c "REINDEX TABLE audit.audit_logs;"
20
21 # 3. Atualizar estatisticas
22 echo "$(date): Atualizando estatisticas" >> $LOGFILE
23 psql -d finops -c "ANALYZE;"
24
25 # 4. Refresh das views materializadas
26 echo "$(date): Atualizando views materializadas" >> $LOGFILE
27 psql -d finops -c "REFRESH MATERIALIZED VIEW CONCURRENTLY
28     mv_resumo_mensal;"
29 psql -d finops -c "REFRESH MATERIALIZED VIEW CONCURRENTLY
30     resumo_financeiro_mensal;"
31
32 # 5. Limpeza de dados antigos
33 echo "$(date): Limpando dados antigos" >> $LOGFILE
34 psql -d finops -c "DELETE FROM audit.audit_logs WHERE
35     timestamp_operacao < CURRENT_DATE - INTERVAL '90 days';"
36 psql -d finops -c "DELETE FROM sessoes WHERE data_expiracao <
37     CURRENT_TIMESTAMP;"
38
39 # 6. Verificar integridade
40 echo "$(date): Verificando integridade" >> $LOGFILE
41 psql -d finops -c "SELECT count(*) as usuarios_inconsistentes FROM
42     usuarios WHERE email IS NULL OR email = '';"
43
44 echo "$(date): Manutencao semanal concluida" >> $LOGFILE
```

Listing 55: Rotina de manutencao semanal

11.3.2 Monitoramento Automatico

```
1 #!/bin/bash
2 # alertas_performance.sh
3
4 DB_NAME="finops"
5 EMAIL_ADMIN="admin@finops.com"
6 LIMITE_CONEXOES=50
7 LIMITE_TEMPO_QUERY=5000 # 5 segundos
```

```

8 LIMITE_CACHE_HIT=95          # 95%
9
10 # Verificar numero de conexoes
11 CONEXOES_ATIVAS=$(psql -d $DB_NAME -t -c "SELECT count(*) FROM
    pg_stat_activity WHERE state = 'active';")
12
13 if [ $CONEXOES_ATIVAS -gt $LIMITE_CONEXOES ]; then
14     echo "ALERTA: Muitas conexoes ativas ($CONEXOES_ATIVAS)" | \
15     mail -s "FinOps Performance Alert" $EMAIL_ADMIN
16 fi
17
18 # Verificar queries lentas
19 QUERIES_LENTAS=$(psql -d $DB_NAME -t -c "
20     SELECT count(*) FROM pg_stat_statements
21     WHERE mean_time > $LIMITE_TEMPO_QUERY AND calls > 10;
22 ")
23
24 if [ $QUERIES_LENTAS -gt 0 ]; then
25     echo "ALERTA: $QUERIES_LENTAS queries com performance ruim" | \
26     mail -s "FinOps Query Performance Alert" $EMAIL_ADMIN
27 fi
28
29 # Verificar cache hit ratio
30 CACHE_HIT=$(psql -d $DB_NAME -t -c "
31     SELECT round(100.0 * sum(blks_hit) / nullif(sum(blks_hit +
    blks_read), 0), 2)
32     FROM pg_stat_database WHERE datname = '$DB_NAME';
33 ")
34
35 if (( $(echo "$CACHE_HIT < $LIMITE_CACHE_HIT" | bc -l) )); then
36     echo "ALERTA: Cache hit ratio baixo ($CACHE_HIT%)" | \
37     mail -s "FinOps Cache Performance Alert" $EMAIL_ADMIN
38 fi
39
40 # Verificar espaco em disco
41 ESPACO_BD=$(psql -d $DB_NAME -t -c "SELECT
    pg_database_size('$DB_NAME');")
42 ESPACO_GB=$((ESPACO_BD / 1024 / 1024 / 1024))
43
44 if [ $ESPACO_GB -gt 10 ]; then
45     echo "AVISO: Banco de dados com ${ESPACO_GB}GB" | \
46     mail -s "FinOps Storage Warning" $EMAIL_ADMIN
47 fi

```

Listing 56: Script de alerta de performance

11.4 Configuracoes de Performance

11.4.1 Parametros Otimizados

```

1 # postgresql.conf - Configuracoes para servidor com 4GB RAM

```

```
2
3 # MEMORIA
4 shared_buffers = 1GB # 25% da RAM
5 effective_cache_size = 3GB # 75% da RAM
6 work_mem = 16MB # Para operacoes complexas
7 maintenance_work_mem = 256MB # Para VACUUM, CREATE INDEX
8
9 # CHECKPOINT E WAL
10 wal_buffers = 16MB
11 checkpoint_completion_target = 0.9
12 max_wal_size = 2GB
13 min_wal_size = 512MB
14 checkpoint_timeout = 10min
15
16 # PLANNER
17 effective_io_concurrency = 200 # Para SSD
18 random_page_cost = 1.1 # Para SSD
19 default_statistics_target = 100
20
21 # CONEXOES
22 max_connections = 100
23 shared_preload_libraries = 'pg_stat_statements'
24
25 # LOG E MONITORAMENTO
26 log_min_duration_statement = 1000 # Log queries > 1s
27 log_checkpoints = on
28 log_connections = on
29 log_disconnections = on
30 log_lock_waits = on
31 log_statement = 'mod'
32 log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
33
34 # AUTO VACUUM
35 autovacuum = on
36 autovacuum_max_workers = 3
37 autovacuum_naptime = 20s
```

Listing 57: Configuracoes recomendadas do PostgreSQL

12 Manutencao e Operacoes

12.1 Rotinas de Manutencao

12.1.1 Manutencao Diaria

```
1 #!/bin/bash
2 # manutencao_diaria.sh
3
4 DB_NAME="finops"
5 LOGFILE="/var/log/finops_daily.log"
```



```
6
7 echo "$(date): Iniciando manutencao diaria" >> $LOGFILE
8
9 # 1. Processar ocorrencias vencidas
10 echo "$(date): Processando ocorrencias" >> $LOGFILE
11 RECORRENCIAS_PROCESSADAS=$(psql -d $DB_NAME -t -c "SELECT
    processar_ocorrencias();" )
12 echo "$(date): $RECORRENCIAS_PROCESSADAS ocorrencias processadas"
    >> $LOGFILE
13
14 # 2. Atualizar saldos das contas
15 echo "$(date): Atualizando saldos" >> $LOGFILE
16 psql -d $DB_NAME -c "
17 DO \$\$
18 DECLARE
19     usuario_record RECORD;
20 BEGIN
21     FOR usuario_record IN SELECT id FROM usuarios WHERE ativo =
        true
22     LOOP
23         CALL atualizar_saldos_usuario(usuario_record.id);
24     END LOOP;
25 END \$\$;
26 "
27
28 # 3. Atualizar metas de poupanca
29 echo "$(date): Atualizando progresso das metas" >> $LOGFILE
30 psql -d $DB_NAME -c "
31 UPDATE metas SET
32     valor_atual = (
33         SELECT COALESCE(SUM(t.valor), 0)
34         FROM transacoes t
35         WHERE t.categoria_id = metas.categoria_id
36             AND t.usuario_id = metas.usuario_id
37             AND t.tipo = 'receita'
38             AND t.status = 'confirmada'
39     ),
40     alcancada = (valor_atual >= valor_alvo),
41     data_alcancada = CASE
42         WHEN valor_atual >= valor_alvo AND data_alcancada IS NULL
43         THEN CURRENT_TIMESTAMP
44         ELSE data_alcancada
45     END
46 WHERE ativa = true;
47 "
48
49 # 4. Limpar sessoes expiradas
50 echo "$(date): Limpando sessoes expiradas" >> $LOGFILE
51 SESSOES_REMOVIDAS=$(psql -d $DB_NAME -t -c "
52     DELETE FROM sessoes
53     WHERE data_expiracao < CURRENT_TIMESTAMP
```

```
54         OR data_ultimo_acesso < CURRENT_TIMESTAMP - INTERVAL '7
           days';
55         SELECT ROW_COUNT();
56     ")
57 echo "$(date): $SESSOES_REMOVIDAS sessoes removidas" >> $LOGFILE
58
59 # 5. Refresh das views materializadas criticas
60 echo "$(date): Atualizando views materializadas" >> $LOGFILE
61 psql -d $DB_NAME -c "REFRESH MATERIALIZED VIEW CONCURRENTLY
           mv_resumo_mensal;"
62
63 echo "$(date): Manutencao diaria concluida" >> $LOGFILE
```

Listing 58: Script de manutencao diaria

12.1.2 Manutencao Mensal

```
1 #!/bin/bash
2 # manutencao_mensal.sh
3
4 DB_NAME="finops"
5 LOGFILE="/var/log/finops_monthly.log"
6
7 echo "$(date): Iniciando manutencao mensal" >> $LOGFILE
8
9 # 1. Arquivar dados antigos
10 echo "$(date): Arquivando dados antigos" >> $LOGFILE
11
12 # Criar tabela de arquivo se nao existir
13 psql -d $DB_NAME -c "
14 CREATE TABLE IF NOT EXISTS arquivo.transacoes_antigas (
15     LIKE transacoes INCLUDING ALL
16 );
17 "
18
19 # Mover transacoes com mais de 2 anos para arquivo
20 psql -d $DB_NAME -c "
21 INSERT INTO arquivo.transacoes_antigas
22 SELECT * FROM transacoes
23 WHERE data_transacao < CURRENT_DATE - INTERVAL '2 years';
24
25 DELETE FROM transacoes
26 WHERE data_transacao < CURRENT_DATE - INTERVAL '2 years';
27 "
28
29 # 2. Recriar indices fragmentados
30 echo "$(date): Recriando indices fragmentados" >> $LOGFILE
31 psql -d $DB_NAME -c "
32 DO \$\$
33 DECLARE
34     idx_record RECORD;
```

```
35 BEGIN
36     FOR idx_record IN
37         SELECT schemaname, tablename, indexname
38         FROM pg_stat_user_indexes
39         WHERE idx_scan > 1000 AND idx_tup_read > idx_tup_fetch * 2
40     LOOP
41         EXECUTE 'REINDEX INDEX ' ||
42         quote_ident(idx_record.indexname);
43     END LOOP;
44 END \$$\$$;
45
46 # 3. Atualizar estatisticas completas
47 echo "$(date): Atualizando estatisticas completas" >> $LOGFILE
48 psql -d $DB_NAME -c "ANALYZE VERBOSE;"
49
50 # 4. Verificar integridade referencial
51 echo "$(date): Verificando integridade referencial" >> $LOGFILE
52 psql -d $DB_NAME -c "
53 -- Verificar orfaos em transacoes
54 SELECT 'Transacoes sem usuario:' as problema, count(*) as
55     quantidade
56 FROM transacoes t
57 LEFT JOIN usuarios u ON t.usuario_id = u.id
58 WHERE u.id IS NULL
59
60 UNION ALL
61
62 SELECT 'Transacoes sem conta:' as problema, count(*) as quantidade
63 FROM transacoes t
64 LEFT JOIN contas c ON t.conta_id = c.id
65 WHERE c.id IS NULL
66
67 UNION ALL
68
69 SELECT 'Contas sem usuario:' as problema, count(*) as quantidade
70 FROM contas c
71 LEFT JOIN usuarios u ON c.usuario_id = u.id
72 WHERE u.id IS NULL AND c.compartilhada = false;
73 "
74
75 # 5. Otimizar tamanho do banco
76 echo "$(date): Executando VACUUM FULL em tabelas grandes" >>
77 $LOGFILE
78 psql -d $DB_NAME -c "
79 SELECT 'VACUUM FULL ' || schemaname || '.' || tablename || ';'
80 FROM pg_tables
81 WHERE schemaname = 'public'
82 AND pg_total_relation_size(schemaname||'.'||tablename) >
83     100*1024*1024; -- >100MB
84 " | psql -d $DB_NAME
```

```
82  
83 echo "$ (date): Manutencao mensal concluida" >> $LOGFILE
```

Listing 59: Script de manutencao mensal

12.2 Monitoramento Operacional

12.2.1 Dashboard de Status

```
1 CREATE OR REPLACE FUNCTION status_sistema()  
2 RETURNS TABLE (  
3     componente TEXT,  
4     status TEXT,  
5     detalhes TEXT,  
6     ultimo_check TIMESTAMP  
7 ) AS $$  
8 BEGIN  
9     -- Status do banco de dados  
10    RETURN QUERY SELECT  
11        'Banco de Dados'::TEXT,  
12        'OK'::TEXT,  
13        'Conectado e funcionando'::TEXT,  
14        CURRENT_TIMESTAMP;  
15  
16    -- Status das conexoes  
17    RETURN QUERY  
18    WITH conexoes AS (  
19        SELECT count(*) as total FROM pg_stat_activity WHERE state  
20    = 'active'  
21    )  
22    SELECT  
23        'Conexoes Ativas'::TEXT,  
24        CASE WHEN total > 80 THEN 'ALERTA' ELSE 'OK' END::TEXT,  
25        'Total: ' || total::TEXT,  
26        CURRENT_TIMESTAMP  
27    FROM conexoes;  
28  
29    -- Status do tamanho do banco  
30    RETURN QUERY  
31    WITH tamanho AS (  
32        SELECT pg_size_pretty(pg_database_size('finops')) as  
33    size_text,  
34        pg_database_size('finops') as size_bytes  
35    )  
36    SELECT  
37        'Tamanho BD'::TEXT,  
38        CASE WHEN size_bytes > 10*1024*1024*1024 THEN 'ATENCAO '  
39    ELSE 'OK' END::TEXT,  
40    size_text,  
41    CURRENT_TIMESTAMP  
42    FROM tamanho;
```

```

40
41  -- Status dos backups
42  RETURN QUERY
43  WITH ultimo_backup AS (
44      SELECT
45          EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP -
46      MAX(timestamp_operacao)))/3600 as horas
47      FROM audit.audit_logs
48      WHERE tabela = 'sistema' AND operacao = 'BACKUP'
49  )
50  SELECT
51      'Ultimo Backup'::TEXT,
52      CASE
53          WHEN horas IS NULL THEN 'ERRO'
54          WHEN horas > 48 THEN 'ATRASADO'
55          ELSE 'OK'
56      END::TEXT,
57      CASE
58          WHEN horas IS NULL THEN 'Nenhum backup encontrado'
59          ELSE 'Ha ' || round(horas::numeric, 1) || ' horas'
60      END,
61      CURRENT_TIMESTAMP
62  FROM ultimo_backup;
63
64  -- Status das recorrências
65  RETURN QUERY
66  WITH recorrências_pendentes AS (
67      SELECT count(*) as pendentes
68      FROM recorrências
69      WHERE ativa = true AND proxima_execucao < CURRENT_DATE
70  )
71  SELECT
72      'Recorrências'::TEXT,
73      CASE WHEN pendentes > 10 THEN 'ATENÇÃO' ELSE 'OK'
74  END::TEXT,
75      pendentes::TEXT || ' pendentes',
76      CURRENT_TIMESTAMP
77  FROM recorrências_pendentes;
78  END;
79  $$ LANGUAGE plpgsql;

```

Listing 60: Funcao para status geral do sistema

12.2.2 Alertas Automaticos

```

1  -- Tabela para armazenar alertas
2  CREATE TABLE sistema.alertas (
3      id BIGSERIAL PRIMARY KEY,
4      tipo VARCHAR(50) NOT NULL,
5      severidade INTEGER NOT NULL, -- 1=info, 2=warning, 3=error,
6      4=critical

```

```
6      titulo VARCHAR(200) NOT NULL,
7      descricao TEXT,
8      usuario_id UUID REFERENCES usuarios(id),
9      resolvido BOOLEAN DEFAULT false,
10     data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11     data_resolucao TIMESTAMP
12 );
13
14 -- Funcao para criar alerta
15 CREATE OR REPLACE FUNCTION criar_alerta(
16     p_tipo TEXT,
17     p_severidade INTEGER,
18     p_titulo TEXT,
19     p_descricao TEXT DEFAULT NULL,
20     p_usuario_id UUID DEFAULT NULL
21 )
22 RETURNS void AS $$
23 BEGIN
24     INSERT INTO sistema.alertas (tipo, severidade, titulo,
25     descricao, usuario_id)
26     VALUES (p_tipo, p_severidade, p_titulo, p_descricao,
27     p_usuario_id);
28
29     -- Se for critico, notificar imediatamente
30     IF p_severidade >= 4 THEN
31         PERFORM pg_notify('alerta_critico',
32             json_build_object(
33                 'tipo', p_tipo,
34                 'titulo', p_titulo,
35                 'usuario_id', p_usuario_id
36             )::text
37         );
38     END IF;
39 END;
40 $$ LANGUAGE plpgsql;
41
42 -- Trigger para detectar transacoes suspeitas
43 CREATE OR REPLACE FUNCTION verificar_transacao_suspeita()
44 RETURNS TRIGGER AS $$
45 DECLARE
46     media_usuario DECIMAL(15,2);
47     limite_alerta DECIMAL(15,2);
48 BEGIN
49     -- Calcular media de transacoes do usuario nos ultimos 30 dias
50     SELECT COALESCE(AVG(valor), 0) INTO media_usuario
51     FROM transacoes
52     WHERE usuario_id = NEW.usuario_id
53           AND tipo = NEW.tipo
54           AND data_transacao >= CURRENT_DATE - INTERVAL '30 days';
55
56     limite_alerta := media_usuario * 5; -- 5x a media
```

```
55
56 -- Criar alerta se valor for muito alto
57 IF NEW.valor > limite_alerta AND limite_alerta > 100 THEN
58     PERFORM criar_alerta(
59         'transacao_alta',
60         2,
61         'Transacao com valor elevado detectada',
62         'Valor: R$ ' || NEW.valor || ' (Media: R$ ' ||
round(media_usuario, 2) || ')',
63         NEW.usuario_id
64     );
65 END IF;
66
67 RETURN NEW;
68 END;
69 $$ LANGUAGE plpgsql;
70
71 CREATE TRIGGER trg_verificar_transacao_suspeita
72 AFTER INSERT ON transacoes
73 FOR EACH ROW EXECUTE FUNCTION verificar_transacao_suspeita();
```

Listing 61: Sistema de alertas baseado em triggers

12.3 Procedures de Operacao

12.3.1 Gerenciamento de Usuarios

```
1 -- Procedure para desativar usuario
2 CREATE OR REPLACE PROCEDURE desativar_usuario(p_usuario_id UUID)
3 LANGUAGE plpgsql AS $$
4 BEGIN
5     -- Desativar usuario
6     UPDATE usuarios SET ativo = false WHERE id = p_usuario_id;
7
8     -- Desativar contas
9     UPDATE contas SET ativa = false WHERE usuario_id =
p_usuario_id;
10
11     -- Encerrar sessoes ativas
12     UPDATE sessoes SET ativa = false WHERE usuario_id =
p_usuario_id;
13
14     -- Log da operacao
15     INSERT INTO audit.audit_logs (tabela, operacao, usuario_id,
dados_novos)
16     VALUES ('usuarios', 'DEACTIVATE', p_usuario_id,
jsonb_build_object('timestamp', CURRENT_TIMESTAMP));
17
18
19     COMMIT;
20 END;
21 $$;
```

```
22
23 -- Procedure para migrar dados entre usuarios
24 CREATE OR REPLACE PROCEDURE migrar_dados_usuario(
25     p_usuario_origem UUID,
26     p_usuario_destino UUID
27 )
28 LANGUAGE plpgsql AS $$
29 BEGIN
30     -- Verificar se usuarios existem
31     IF NOT EXISTS (SELECT 1 FROM usuarios WHERE id =
32 p_usuario_origem) THEN
33         RAISE EXCEPTION 'Usuario origem nao encontrado';
34     END IF;
35
36     IF NOT EXISTS (SELECT 1 FROM usuarios WHERE id =
37 p_usuario_destino) THEN
38         RAISE EXCEPTION 'Usuario destino nao encontrado';
39     END IF;
40
41     -- Migrar transacoes
42     UPDATE transacoes SET usuario_id = p_usuario_destino
43     WHERE usuario_id = p_usuario_origem;
44
45     -- Migrar contas (marcar como transferidas)
46     UPDATE contas SET
47         usuario_id = p_usuario_destino,
48         nome = nome || ' (Transferida)'
49     WHERE usuario_id = p_usuario_origem;
50
51     -- Migrar categorias personalizadas
52     UPDATE categorias SET usuario_id = p_usuario_destino
53     WHERE usuario_id = p_usuario_origem AND sistema = false;
54
55     -- Log da migracao
56     INSERT INTO audit.audit_logs (tabela, operacao, dados_novos)
57     VALUES ('usuarios', 'MIGRATE',
58         jsonb_build_object(
59             'origem', p_usuario_origem,
60             'destino', p_usuario_destino,
61             'timestamp', CURRENT_TIMESTAMP
62         ));
63
64     COMMIT;
65 END;
66 $$;
```

Listing 62: Procedures para operacoes de usuario

12.4 Automacao e Jobs

12.4.1 Configuracao de Jobs

```
1 # /etc/cron.d/finops
2
3 # Manutencao diaria (2:00 AM)
4 0 2 * * * postgres /opt/finops/scripts/manutencao_diaria.sh
5
6 # Backup diario (3:00 AM)
7 0 3 * * * postgres /opt/finops/scripts/backup_completo.sh
8
9 # Verificacao de backup (3:30 AM)
10 30 3 * * * postgres /opt/finops/scripts/verificar_backups.sh
11
12 # Manutencao semanal (domingo 4:00 AM)
13 0 4 * * 0 postgres /opt/finops/scripts/manutencao_semanal.sh
14
15 # Manutencao mensal (primeiro domingo 5:00 AM)
16 0 5 1-7 * 0 postgres /opt/finops/scripts/manutencao_mensal.sh
17
18 # Monitoramento de performance (a cada 15 minutos)
19 */15 * * * * postgres /opt/finops/scripts/alertas_performance.sh
20
21 # Processamento de recorrencias (todo dia 1:00 AM)
22 0 1 * * * postgres psql -d finops -c "SELECT
23     processar_recorrencias();"
24
25 # Limpeza de logs (semanal, sabado 23:00)
26 0 23 * * 6 postgres /opt/finops/scripts/limpeza_logs.sh
```

Listing 63: Configuracao de cron jobs